



Instituto Politécnico Nacional
Escuela Superior de Cómputo



Práctica 04.

Simulaciones con el TAD Cola

Maestro: Edgardo Adrián Franco Martínez



Alumnos:

Erendil Aguilar Avendaño

Iker Itzae Aguilar Souza



Grupo 2CM2

Ciudad de México, 09 de junio de 2024.

Índice

Introducción	3
Colas	3
Planteamiento de los problemas	4
Simulación 01: Supermercado	4
Simulación 02: Ejecución de procesos en el sistema operativo.	4
Simulación 03: Banco	4
Diseño y funcionamiento de la solución	5
Simulación 01: Supermercado	5
Simulación 02: Ejecución de procesos en el sistema operativo	6
Simulación 03: Banco	7
Implementación de la solución.....	8
Simulación 01: Supermercado	8
Simulación 02: Ejecución de procesos en el sistema operativo.	10
Simulación 03: Banco	13
Funcionamiento	16
Simulación 01: Supermercado	16
Simulación 02: Ejecución de procesos en el sistema operativo	17
Simulación 03: Banco	18
Conclusiones	19
Anexos.....	20
Simulación 01: Supermercado	20
Simulación 02: Ejecución de procesos en el sistema operativo.	30
Simulación 03: Banco	41
Bibliografía	50

Introducción

Colas

Una cola en programación es una estructura de datos que sigue el principio de FIFO (First In First Out), lo que significa que el primero en entrar será el primero en salir, en otras palabras, los elementos se insertan al final de la cola y las supresiones son al frente de la cola.

Un subdesbordamiento ocurre cuando se trata de desencolar o acceder a elementos de una cola vacía. Por otro lado, un desbordamiento ocurre cuando se trata de encolar elementos a una cola que ha llegado a su máximo tamaño.

En una cola no se puede ver o desencolar elementos. Antes de sacar un elemento se verifica si la cola está vacía.

Una cola tiene tres tipos de implementaciones:

Estática: es cuando se le asigna una cantidad fija de memoria antes de la ejecución de un programa. La cantidad de espacio asignado se determina durante la compilación y no varía al ejecutarse.

Estática circular: es similar a la estática, pero no tiene extremos, el frente es el mismo que el final.

Dinámica: es cuando se le asigna memoria a medida que se necesita, durante la ejecución del programa. La memoria no queda fija durante la compilación.

Colas de prioridad.

A los elementos se les asigna una prioridad de modo que son extraídos de acuerdo con el elemento de mayor prioridad. Si dos elementos tienen la misma prioridad son procesados en el orden en que fueron introducidos.

En una cola de prioridad el orden de atención no está dado solo por el orden de llegada, cada elemento tiene asociado una prioridad y cada uno será procesado según su prioridad.

Planteamiento de los problemas

En esta práctica se plantean tres problemas a resolver:

Simulación 01: Supermercado

Simular la atención de clientes en un supermercado, el cual deberá de atender al menos a 100 clientes por día para no tener pérdidas, por lo que, una vez que se hayan atendido a más de 100 personas y no quede gente formada en las cajas la tienda podrá cerrar. Mientras no se cierre la tienda, las personas podrán seguir llegando con productos a las cajas.

Simulación 02: Ejecución de procesos en el sistema operativo.

Simular la ejecución de los procesos gestionados por el sistema operativo en un equipo monoprocesador sin manejo de prioridades.

Se maneja únicamente el cambio de la cola de listos a ejecución y cuando termina el proceso se envía a la cola de terminados.

Simulación 03: Banco

Simular la atención de personas en un banco, se deben respetar las políticas de atención y las personas siempre deben estar atendidas.

Diseño y funcionamiento de la solución

Simulación 01: Supermercado

Entrada de la simulación

- ✚ Nombre del supermercado (sin espacios)
- ✚ Numero de cajeros que lo atenderán $0 < n < 11$
- ✚ Tiempos en milisegundos de atención de cada cajera (Deberán ser múltiplos de 10 ms; pg. 10, 20, 30, ...)
- ✚ Tiempos en milisegundos de llegada de los compradores a las cajas (Deberán ser múltiplos de 10 ms igualmente)

Salida de la simulación (Pantalla)

- ✚ Llegada de los clientes a las colas de las cajas.
- ✚ Clientes en espera de cada cola
- ✚ Cliente que es atendido en cada caja
- ✚ Número de clientes atendidos en su totalidad
- ✚ Nombre de la tienda y anuncio de cierre

Detalles

- ✚ Al iniciar el número de clientes en cada fila es 0.
- ✚ Número de cajas $0 < n < 11$
- ✚ Los identificadores de los clientes son un número consecutivo único para cada uno.
- ✚ Cuando un cliente llega a formarse, este selecciona aleatoriamente una cola para formarse y no cambia de cola hasta que es atendido.
- ✚ Para que pueda cerrarse la tienda deben haberse atendido al menos a 100 clientes, y ya no debe haber nadie formado en las colas de las cajas.

Simulación 02: Ejecución de procesos en el sistema operativo

Entrada a la simulación

- La cantidad de procesos en la cola y sus propiedades: Nombre, actividad, ID, tiempo.
- Tabla de datos a recibir para cada proceso (Nombre y Actividad son cadenas que pueden incluir espacios)

Nombre del proceso	Actividad	ID	de procesador Tiempo (Segundos)
char[45]	char[200]	char[45]	int

Ejemplo

Nombre del proceso	Actividad	ID	Tiempo (Segundos)
Microsof Word	Procesamiento de textos con formato	001W01	30

Salida de la simulación (Pantalla)

- Proceso en ejecución actual y sus datos (Nombre, ID, Actividad y Tiempo total que lleva ejecutándose) ->Tiempo en la cola de listos + tiempo de ejecución total.
- ID y Nombre del último proceso ejecutado y el tiempo de procesador que falta para que este proceso concluya.
- ID y Nombre del proceso siguiente a ser ejecutado y el tiempo que falta para que este proceso concluya.
- Cuando un proceso termina este se coloca en la cola de finalizados almacenando su tiempo total (Tiempo en la cola de listos + tiempo de ejecución total).
- Cuando terminen todos los procesos mostrar en el orden de finalización el Nombre, ID y tiempo total que tardo en terminar cada proceso.

Detalles

- El Tiempo de cada Quantum de tiempo para despacharlos es de 1 segundo.
- El proceso que se encontraba en ejecución se Encola y se coloca al proceso Desencolado en ejecución.



Simulación 03: Banco

Políticas de atención

El banco cuenta con entre 1 y 10 cajas en operación las cuales atienden a tres filas (Clientes, Usuarios y Preferentes).

- Los clientes del banco (personas con cuenta en ese banco), son atendidos por cualquier cajero y nunca dejan de ser atendidos por alguna caja.
- Los usuarios del banco (personas sin cuenta en ese banco), son atendidos según la disponibilidad de alguna caja, nunca permitiendo que pasen más de 5 personas de las otras dos filas sin que una persona de esta fila sea atendida.
- Los clientes preferentes (personas con más de una cuenta en ese banco y privilegios preferenciales), serán atendidos por cualquier cajero disponible con mayor prioridad que a los clientes y usuarios.

Entrada a la simulación

- Número de cajeros en el banco Número $0 < n < 11$
- Tiempo de atención en milisegundos de cada cajero (Múltiplos de 10 ms)
- Tiempo de llegada de los clientes del banco
- Tiempo de llegada de los usuarios del banco
- Tiempo de llegada de los clientes preferentes

Salida de la simulación

- Llegada de los clientes a las 3 filas del banco
- Clientes en espera de cada fila
- Cliente que es atendido en cada caja y su tipo (Cliente, preferente o usuario)
- Cajeros sin realizar ninguna atención en caso de que así sea.

Detalles de la simulación 03

- Al iniciar el número de personas en cada fila es 0
- Número de cajeros $0 < n < 11$
- Los identificadores de las personas son un número consecutivo único por tipo.
 - P1(Preferente 1), C1 (Cliente 1), U1 (Usuario 1), P2, P3, etc.
- Importante considerar los tiempos dados en múltiplos de 10ms.
- El banco nunca cierra

Implementación de la solución

Cada programa se dividió en varias funciones para resolver los problemas de manera precisa.

Simulación 01: Supermercado

Pedir Requisitos

Con la función **PedirRequisitos** se le pide al usuario ingresar los datos del supermercado: el nombre del súper, el número de cajeros que atenderán, el tiempo que tardará cada cajero en atender a los clientes, y el tiempo de llegada de los clientes. Los tiempos se toman como múltiplos de 10 (por ejemplo: 10 ms).

Si el nombre de la tienda tiene más de 50 caracteres, el número de cajas es menor a 0 o mayor a 10, o los tiempos ingresados no son múltiplos de 10, el programa le volverá a pedir al usuario que escriba correctamente el nombre de la tienda, el número de cajas y los tiempos, respectivamente.

Simulación

Luego comienza la simulación del supermercado con la llamada a la función **Simulacion**.

Se declaran las variables:

- ✚ **tiempo** es el contador de tiempo de la simulación,
- ✚ **clienteN** es el contador del número de clientes que llegan al súper
- ✚ **atendidos** es el contador del número de clientes atendidos,
- ✚ **i, filaC, aux** son variables auxiliares
- ✚ **e** es una variable de tipo *elemento* para manejar clientes en la cola.

Se inicializa la función **rand** que genera números aleatorios con el tiempo actual.

Se crean e inicializan las colas:

- ✚ **cajeras** es un arreglo de colas en donde cada una representa la fila de una caja.
- ✚ **tiempoC** almacena el tiempo de atención para cada caja.
- ✚ **atendidosC** almacena el número de clientes atendidos por cada caja.

Llama a la función **AbrirTienda** para dibujar el marco inicial de la simulación.

Empieza un ciclo infinito de la simulación con **While**:

Se espera un tiempo antes de empezar con la función **EsperarMiliSeg(TIEMPO_BASE)** e incrementa **tiempo**.

Atención a clientes

Mediante un ciclo **for** se atienden a los clientes que lleguen. Verifica si el tiempo **tiempo** es múltiplo del tiempo de atención de alguna caja (**cajasT[i]**). Si lo es entonces atiende a un cliente de esa caja llamando a **QuitarCliente**, e incrementa los contadores **atendidos** y **atendidosC[i]**. Y actualiza el número de clientes atendidos en pantalla.

Cierre de la tienda

Si el número total de clientes atendidos es mayor o igual a 100, verifica si quedan clientes en las colas.

Si no quedan clientes, llama a **CerrarTienda** y termina el ciclo.

Llegada de Clientes:

Si el **tiempo** es múltiplo del tiempo de llegada de clientes (**clienteT**), incrementa el contador de clienteN.

Selecciona aleatoriamente una caja (**filaC**) para el nuevo cliente.

Si la caja seleccionada está vacía, calcula el tiempo restante hasta que esa caja pueda atender al cliente.

Llama a **AgregarCliente** para añadir el nuevo cliente a la caja seleccionada.

Funciones de modificación:

-  **AgregarCliente:** es una función que calcula la posición en la pantalla donde se encuentra la caja, actualiza la representación visual de la cola en la pantalla y espera un tiempo antes de agregar el cliente a la cola.
-  **QuitarCliente:** es una función que calcula la posición en la pantalla donde se encuentra la caja, remueve un cliente de la cola de la cajera, actualiza la representación visual de la cola en la pantalla y retorna el cliente eliminado.
-  **AbrirTienda:** es una función que muestra gráficamente la apertura del supermercado. Incluye una presentación, una animación del marco, de las cajas, los estantes, y un cartel inicial con los nombres de los autores.
-  **CerrarTienda:** es una función que muestra gráficamente el cierre del supermercado. Incluye un anuncio de cierre, una animación de cortina, y un cartel final con datos de la operación de la tienda.

Las siguientes funciones dibujan gráficamente el supermercado en la pantalla:

-  **DibujaPresentacion:** introducción a la simulación del súper.
-  **DibujaMarco:** dibuja el marco de la ventana del súper en la pantalla.

-  **DibujaCajas:** dibuja las cajas del súper en la pantalla, mostrando visualmente las ubicaciones de las cajas disponibles para el servicio.
-  **DibujaEstantes:** dibujar los estantes del súper en la pantalla, mostrando visualmente los productos que están disponibles en la tienda.
-  **DibujaCortina:** animación de una cortina en la pantalla, para apertura y cierre del súper.
-  **DibujaCartel:** dibuja un cartel en la pantalla.
-  **DibujaCartelDatos:** muestra datos de súper como: nombre, número de cajas, cantidad de clientes atendidos.
-  **DibujaAnuncioAbrir:** dibuja un anuncio visual en la pantalla para indicar que el súper está abierto.
-  **DibujaAnuncioCerrar:** dibuja un anuncio visual en la pantalla para indicar que el súper está cerrado.

Simulación 02: Ejecución de procesos en el sistema operativo.

Pedir Requisitos

Con la función **PedirRequisitos** se le pide al usuario ingresar los datos sistema operativo: el número de procesos a ejecutar, el nombre de cada uno de los procesos, una breve descripción de lo que realizará el proceso, y el tiempo de cada proceso.

En esta función se crea un ID para identificar cada proceso, contiene una parte numérica (basada en el índice *i*), dos caracteres significativos extraídos del nombre del proceso, y un número de ocurrencia para manejar procesos con nombres duplicados.

Los tiempos se toman como múltiplos de 10 (por ejemplo: 10 ms).

Simulación

Comienza la simulación del supermercado con la llamada a la función **Simulacion**.

Se declaran las variables:

-  **tiempo:** Contador de tiempo de la simulación.
-  **elemento e:** Estructura que representa un proceso.

Se inicializa la función rand que genera números aleatorios con el tiempo actual.

Se declaran colas y se inicializan:

-  **Ejecucion, Finalizados:** Colas para gestionar procesos en ejecución y finalizados.

Se inicia el sistema operativo con la función *AbrirSistemaOperativo*.

Empieza un ciclo infinito que se interrumpe cuando todas las colas de procesos están vacías. Incrementa el tiempo y se checa el estado del proceso en ejecución. Si hay procesos en la cola de ejecución se obtiene el primer proceso y se verifica si se ha terminado.

Cuando un proceso ha terminado se mueve a la cola de finalizados y se llama a *TerminarProceso*. Por otro lado, si no ha terminado se mueve a la cola de faltantes y se llama a *QuitarProceso*.

Si aún hay procesos por hacer se obtiene y actualiza el primer proceso, y se agrega el proceso a la cola de ejecución llamando a *AgregarProceso*.

Se espera un tiempo base (1 segundo) con *EsperarMiliSeg*. Luego se verifican si ambas colas (faltantes y ejecución) están vacías, si lo están se cierra el sistema operativo y termina la simulación.

 **AgregarProceso:** representa visualmente el movimiento y la gestión de procesos en el sistema operativo simulado. Gestiona la visualización del proceso de agregar un elemento desde **Faltantes** a **Ejecución**, en donde se elimina cualquier elemento previamente visualizado en la posición de **Faltantes**, visualiza el elemento que será agregado desde **Faltantes** o **Ejecución** y espera un breve momento para la animación. Elimina cualquier flecha preexistente en la pantalla para limpiar el área, anima el proceso de agregar un elemento dibujando una flecha que indica el movimiento del proceso y dibuja el nuevo estado del proceso en la cola de **Ejecución** después de agregarlo. Incluye tiempos de espera entre cada paso para hacer la animación visible.

 **QuitarProceso:** representa visualmente la gestión y eliminación de procesos en el sistema operativo simulado. Gestiona la visualización de la eliminación de un proceso desde la cola **Faltantes**, en donde elimina cualquier elemento previamente visualizado en la posición 0, espera un breve momento para la animación, elimina cualquier flecha preexistente en la pantalla para limpiar el área antes de la animación, anima el proceso de eliminar un elemento dibujando una flecha que indica el movimiento del proceso y dibuja el nuevo estado de la cola **Faltantes** después

de eliminar el proceso, actualizando la visualización con el último elemento de la cola en la posición adecuada. Incluye tiempos de espera entre cada paso para hacer la animación visible.

- ✚ **TerminarProceso:** representa la animación de la finalización de procesos en el sistema operativo simulado, ayudando a visualizar el movimiento de los procesos a medida que se completan.
Gestiona la visualización de la finalización de un proceso, en donde elimina cualquier elemento previamente visualizado en la posición 0, condicionalmente elimina el elemento en la posición 1 si la cola **Faltantes** está vacía, espera un breve momento para la animación, elimina cualquier flecha preexistente en la pantalla para limpiar el área antes de la animación, anima el proceso de agregar una flecha que indica el movimiento del proceso desde **Faltantes** a **Finalizados** y dibuja el nuevo estado de la cola **Finalizados** después de finalizar el proceso, actualizando la visualización con el último elemento de la cola en la posición adecuada.
Incluye tiempos de espera entre cada paso para hacer la animación visible.

- ✚ **AbrirSistemaOperativo:** es la configuración inicial de la interfaz gráfica. Inicializa las estructuras de datos y se visualiza un anuncio de apertura del sistema operativo.

- ✚ **CerrarSistemaOperativo:** maneja el cierre ordenado de la simulación del sistema operativo. Se visualiza un anuncio de cierre del sistema operativo mostrando los datos finales de los procesos que han terminado.

- ✚ **DibujaPresentacion:** introducción a la simulación del sistema operativo.
- ✚ **DibujaMarco:** dibuja el marco de la ventana del S.O. en la pantalla
- ✚ **DibujaCuadros:** dibuja cuadros específicos de los procesos o el estado del sistema.
- ✚ **DibujaElementoAgregar:** Visualiza la adición de un nuevo elemento (proceso) a una cola o área de la interfaz. El parámetro tipo puede indicar en qué cola o sección se está agregando el elemento.
- ✚ **DibujaElementoEliminar:** Visualiza la eliminación de un elemento (proceso) de una cola o área de la interfaz. El parámetro tipo puede indicar de qué cola o sección se está eliminando el elemento.

- ✚ **DibujaCortina:** animación de una cortina en la pantalla, para apertura y cierre del sistema.
- ✚ **DibujaCartel:** dibuja un cartel en la pantalla.
- ✚ **DibujaCartelDatos:** muestra datos de los procesos finalizados.
- ✚ **DibujaAnuncioAbrir:** dibuja un anuncio visual en la pantalla para indicar la apertura del sistema operativo.
- ✚ **DibujaAnuncioCerrar:** dibuja un anuncio visual en la pantalla para indicar el cierre del sistema operativo.

Simulación 03: Banco

Pedir Requisitos

Con la función **PedirRequisitos** se le pide al usuario ingresar los datos del banco: el número de cajeros, el tiempo de atención, el tiempo de llegada de cada cliente, de los usuarios del banco y de los usuarios preferentes.

Los tiempos se toman como múltiplos de 10 (por ejemplo: 10 ms).

Simulación

Comienza la simulación del supermercado con la llamada a la función **Simulacion**.

Parámetros de entrada:

- ✚ **cajeros:** Número de cajeros en el banco.
- ✚ **cajerosT:** Tiempo de atención de cada cajero.
- ✚ **cajasB[]:** Arreglo que indica qué cajas están siendo ocupadas.
- ✚ **clienteC:** Tiempo de llegada de un Cliente.
- ✚ **clienteU:** Tiempo de llegada de un Usuario.
- ✚ **clienteP:** Tiempo de llegada de un Cliente Preferente.

Variables:

- ✚ **tiempo:** Contador de tiempo global para la simulación.
- ✚ **clientesN[]:** Array para contar el número de clientes de cada tipo (Cliente, Usuario, Preferente).
- ✚ **atendidos:** Contador total de clientes atendidos.
- ✚ **clientesT[]:** Array que almacena los tiempos de llegada de cada tipo de cliente.
- ✚ **controlC, controlU:** Variables de control para la verificación de reglas en las filas.
- ✚ **cajerosC[]:** Array de colas para cada cajero.
- ✚ **tiempoC[]:** Array para el tiempo restante de atención de cada cajero.

✚ **atendidosC[]**: Array para el número de clientes atendidos por cada cajero.

Se inicializa la función rand que genera números aleatorios con el tiempo actual.

Se declaran colas y se inicializan:

✚ **filas[3]**: Array de tres filas para los tres tipos de clientes.

✚ **cajerosC[cajeros]**: Array para cada cajero.

Se inicia la simulación del banco con la función **AbrirBanco**. Se espera un tiempo base (1 segundo) con **EsperarMiliSeg**.

Empieza un ciclo infinito que simula el paso del tiempo.

Atención de Clientes

Cada vez que el tiempo es múltiplo del tiempo de atención de los cajeros (**cajerosT**), se atiende a un cliente en cada cajero disponible.

Llegada de Clientes

Los clientes, usuarios y preferentes llegan a sus respectivas filas a intervalos regulares definidos por **clientesT[]**. Se incrementa el número de clientes y se agrega **Cliente** a la **Fila**.

Asignación de Clientes a Cajas

Se verifica si hay cajeros disponibles y se asignan clientes de las filas a las cajas. La asignación de clientes a cajas se hace siguiendo un orden de prioridad: primero los preferentes, luego los clientes y finalmente los usuarios.

Verifica si hay cajas para nuevos clientes, usuarios o preferentes. Si las hay entonces se agregan los clientes a sus respectivas cajas.

✚ **AgregarClienteCaja**: agrega un cliente de una fila específica a una caja específica.

✚ **QuitarClienteCaja**: quita un cliente de una caja específica después de que ha sido atendido.

✚ **AgregarClienteFila**: agrega un cliente a una fila específica.

✚ **QuitarClienteFila**: quita un cliente de la fila de usuarios preferentes después de que ha sido atendido.



✚ **AbrirBanco**: inicializa el estado de las cajas como abiertas y muestra la presentación del banco en la pantalla.



 **DibujaPresentacion:** introducción a la simulación del banco.



 **DibujaMarco:** dibuja el marco de la ventana del súper en la pantalla.



 **DibujaCajas:** dibuja las cajas del banco en la pantalla, mostrando visualmente su estado, si están abiertas o cerradas.



 **DibujaFilas:** dibuja las filas de clientes, usuarios y preferentes en la pantalla.



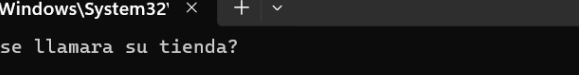
 **DibujaEstantes:** dibuja los estantes en la pantalla.



 **DibujaAnuncioAbrir:** dibuja un anuncio visual en la pantalla para indicar que el banco está abierto.

Funcionamiento

Simulación 01: Supermercado



```
C:\Windows\System32' x + v
¿Como se llamara su tienda?
OXXO
¿Cuantos cajeros quiere que su tienda tenga?
3
¿Cunto tiempo se tardara cada cajero en atender a los clientes?
10
20
30
¿Cual es el tiempo de llegada de los clientes?
40
```

[illegible]

```
C:\Windows\System32' x + v

"0XX0"

Tiempo abierta:    4030 Milisegundos

Clientes Atendidos:    100 Clientes

Cajas mas eficientes:    C3

Clientes Atendidos por Caja:
C1: 28
C2: 29
C3: 43

C:\Users\earen\OneDrive\Documentos\ESCOM\2º semestre\Algoritmos y Estructuras de Datos\Unidad 2\Practica 04>
```

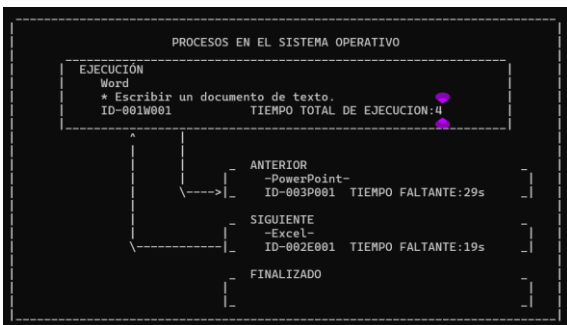

Simulación 02: Ejecución de procesos en el sistema operativo

```

¿Cuántos procesos quiere que se ejecuten en el sistema operativo?
3
¿Como se llamara el proceso 1?
Word
¿Cual es la actividad el proceso 1?
Escribir un documento de texto
¿Cual es el tiempo de proceso para el proceso 1?
10

¿Como se llamara el proceso 2?
Excel
¿Cual es la actividad el proceso 2?
Crear una base de datos
¿Cual es el tiempo de proceso para el proceso 2?
20

¿Como se llamara el proceso 3?
PowerPoint
¿Cual es la actividad el proceso 3?
Diseñar una presentación.
¿Cual es el tiempo de proceso para el proceso 3?
30
    
```



Simulación 03: Banco

```
¿Cuántos cajeros quiere que el Banco tenga?  
4  
¿Cuál es el tiempo de atención de cada cajero?  
10  
¿Cuál es el tiempo de llegada de los clientes del banco?  
20  
¿Cuál es el tiempo de llegada de los usuarios del banco?  
30  
¿Cuál es el tiempo de llegada de los usuarios preferentes?  
40
```

Erendil Aguilar Avendaño

Iker Itzae Aguilar Souza

PRESENTAN

SIMULACION 03

```

" Banco Nacional de México "
Num. Clientes Atendidos: 23      Ultimo movimiento: C1 - C11

+-----+-----+-----+-----+-----+-----+-----+-----+
| C1 | / / / / | C2 | / / / / | / / / / | C3 | C4 | / / / / | / / / / |
+-----+-----+-----+-----+-----+-----+-----+-----+
| - | - | - | - | - | - | - | - |
+-----+-----+-----+-----+-----+-----+-----+-----+
| [#####] | | | C12 | | | | | | | [#####] | |
| [#####] | | | | | | | | | | | [#####] |
| [#####] | | | | | | | | | | | [#####] |
| [#####] | | | | | | | | | | | [#####] |
| [#####] | | | | | | | | | | | [#####] |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

[illegible]

Nota: Este programa nunca termina.

Conclusiones

Aguilar Avendaño Erendil

En esta práctica, que constó de tres ejercicios, pudimos aplicar las tres implementaciones que tiene el TADCola (estática, estática circular y dinámica) en actividades que ocurren cotidianamente y que realmente nadie se pregunta cómo funcionan y no se toma en cuenta que hay todo un algoritmo de programación para realizarlos de tal forma que se vuelva más fácil y que pasen inadvertidos. Éste trabajo fue un reto porque no solo se pusieron en práctica los algoritmos, sino también fue necesario representarlo gráficamente, fue como dibujar el programa.

Aguilar Souza Iker Itzae

En la elaboración de esta práctica, pude observar formas muy creativas para usar un TADCola, con ejemplos prácticos que muchas veces son vistos en la vida cotidiana pero normalmente no se analizan a profundidad. Sin embargo, la enseñanza más significativa de esta práctica es como se pueden mostrar programas de c desde la consola de una manera creativa y bien estructurada para que sea visualmente agradable y dinámico.

Anexos

Simulación 01: Supermercado

```
/*
Autores:  Erendil Aguilar Avendaño
          Iker Itzae Aguilar Souza
Versión 1.6 (03 de Junio 2024)
Grupo:    2CM2
Materia:  Algoritmos y Estructuras de Datos
Práctica 04: Simulaciones con el TAD Cola

=== Simulaciones con el TAD Cola ===

Descripción: Programa que con el uso de colas simula el comportamiento de un Supermercado de manera animada.

Observaciones: El programa requiera de la librería "presentacion.h", la cuál tiene las implementaciones
para mover el cursor de la pantalla, esperar un tiempo y borrar pantalla, así como las librerías "TADColaEst.h"
y "TADCola/TADColaDin.h" para el uso correcto de las colas, la compilación debiera incluir las definiciones de
las funciones según la plataforma que se este utilizando (Windows o Linux).

Compilación: gcc -o simulacion_01 simulacion_01.c presentacion/presentacion(Win|Lin).o
TADCola/TADCola(Din|Est|EstCirc).o (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
objeto"
gcc -o simulacion_01 simulacion_01.c presentacion/presentacion(Win|Lin).c
TADCola/TADCola(Din|Est|EstCirc).c (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
fuente"
Ejecución: Windows simulacion_01.exe & Linux ./simulacion_01
*/

//LIBRERIAS
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include <time.h>
#include <math.h>
#include <locale.h>
#include "presentacion/presentacion.h"
//#include "TADCola/TADColaEst.h" //Si se usa la implemtenentación estática (TADColaEst.c|TADColaEstCirc.c)
#include "TADCola/TADColaDin.h" //Si se usa la implemtenentación dinámica (TADColaDin.c)

//CONSTANTES
#define ALTO 24 //Se piensa en un pantalla de 24 filas x 79 columnas
#define ANCHO 79
#define TIEMPO_BASE 10 //Tiempo base en milisegundos

//FUNCIONES
void PedirRequisitos(char *tienda, int *cajas, int *cajasT, int *clienteT);

void Simulacion(char tienda[], int cajas, int cajasT[], int clienteT);

void AgregarCliente(cola *cajera, int cajas, int clienteN, int filaC);
elemento QuitarCliente(cola *cajera, int cajas, int filaC);

void AbrirTienda(char tienda[], int cajas);
void CerrarTienda(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo);

void DibujaPresentacion();
void DibujaMarco(char tienda[]);
void DibujaCajas(int cajas);
void DibujaEstantes(int cajas);
void DibujaCortina();
void DibujaCartel();
void DibujaCartelDatos(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo);
void DibujaAnuncioAbrir();
void DibujaAnuncioCerrar();

//PROGRAMA PRINCIPAL
int main(void)
{
    char tienda[52];
    int cajas, cajasT[10], clienteT;

    setlocale(LC_ALL, "");

    /***PEDIR REQUISITOS***/
    PedirRequisitos(tienda, &cajas, cajasT, &clienteT);

    /***INCIAR LA TIENDA***/
    Simulacion(tienda, cajas, cajasT, clienteT);

    MoverCursor(ANCHO,ALTO);
}

/*
void PedirRequisitos(char *tienda, int *cajas, int *cajasT, int *clienteT)
Recibe: char * Referencia/Dirección a la cadena del nombre de la tienda, int * Referencia/Dirección al numero
de cajas,
```

```

int * Referencia/Dirección al arreglo de los tiempos de cajas, int *
Referencia/Dirección al tiempo de llegada del cliente
Devuelve: void (No retorna valor explicito)
Observaciones: Función que pide los datos como el nombre de la tienda, el numero de cajas, tiempo de
consulta de cajas y el tiempo de llegada de los clientes, poniendo los resultados en las variables
dadas.
*/
void PedirRequisitos(char *tienda, int *cajas, int *cajasT, int *clienteT){
    int i;
    int Acajas, AclienteT;
    char ch, Atienda[52], c;
    char correcto = 0;

    BorrarPantalla();

    /**Pedir el Nombre de la Tienda***/
    printf("¿Como se llamara su tienda?\n");
    do{
        fgets(Atienda, sizeof(Atienda), stdin);
        // Verificar si la longitud de la cadena ingresada excede 50 caracteres (sin contar el '\n' de fgets)
        if (strlen(Atienda) == 51 && Atienda[50] != '\n') {
            printf("El nombre de la tienda tiene más de 50 caracteres.\n");
            while (getchar() != '\n');
            //exit(1);
        } else {
            //En caso de un salto de linea
            if (Atienda[strlen(Atienda) - 1] == '\n') {
                Atienda[strlen(Atienda) - 1] = '\0';
            }
            correcto = 1;
        }
    }while(correcto == 0);

    /**Pedir el numero de cajeros***/
    printf("¿Cuántos cajeros quiere que su tienda tenga?\n");
    do{
        scanf("%d", &Acajas);
        if(Acajas<1 || Acajas>10){
            printf("Numero de cajas no valido (0<N<11).\n");
        }
    }while(Acajas<1 || Acajas>10);

    /**Pedir los tiempos de consulta de los cajeros***/
    printf("¿Cuánto tiempo se tardara cada cajero en atender a los clientes?\n");
    for(i=0;i<Acajas;i++){
        do{
            scanf("%d", &cajasT[i]);
            if(cajasT[i]<10 || cajasT[i]%10 != 0){
                printf("Tiempo invalido\n");
            }
        }while(cajasT[i]<10 || cajasT[i]%10 != 0);
    }

    /**Pedir el tiempo de llegada de los clientes***/
    printf("¿Cual es el tiempo de llegada de los clientes?\n");
    do{
        scanf("%d", &AclienteT);
        if(AclienteT<10 || AclienteT%10 != 0){
            printf("Tiempo invalido\n");
        }
    }while(AclienteT<10 || AclienteT%10 != 0);

    strncpy(tienda, Atienda, 52);
    *cajas = Acajas;
    *clienteT = AclienteT;
}

/*
void Simulacion(char tienda[], int cajas, int cajasT[], int clienteT)
Recibe: char Cadena del nombre de la tienda, int Numero de cajas, int Arreglo de los tiempos de cajas, int
Tiempo de llegada del cliente
Devuelve: void (No retorna valor explicito)
Observaciones: Función que simulara el comportamiento de un supermercado con base en las variables dadas por
el usuario, permitiendo la llegada de clientes a colas, siendo atendidos en
cajas para llegar a la cantidad de 100 clientes para que pueda parar la simulación, solo si
no hay mas clientes en las colas.
*/
void Simulacion(char tienda[], int cajas, int cajasT[], int clienteT){
    int tiempo = 0, clienteN = 0, atendidos = 0;
    int i, filaC, aux;
    elemento e;

    //Inicializar la función Rand
    srand(time(NULL));

    //Crear las colas
    cola cajeras[cajas];
    int tiempoC[cajas];
    int atendidosC[cajas];

    //Inicializar colas

```

```

for(i=0;i<cajas;i++){
    Initialize(&cajeras[i]);
    tiempoC[i] = 0;
    atendidosC[i] = 0;
}

//Iniciar Tienda
AbrirTienda(tienda, cajas);

//Ciclo infinito de la simulación
while(1){
    EsperarMiliSeg(TIEMPO_BASE); //Esperar el tiempo base
    tiempo++; //Incrementar el contador de tiempo

    //Si el tiempo es multiplo del tiempo de atencion de alguna caja
    for(i=0;i<cajas;i++){
        if ((tiempo+tiempoC[i]) % cajasT[i] == 0){
            if (!Empty(&cajeras[i])){
                //Quitar Cliente de la Caja
                e = QuitarCliente(&cajeras[i], cajas, i);
                atendidos++;
                atendidosC[i]++;
                MoverCursor(30,4);
                printf("%d", atendidos);
                MoverCursor(62,4);
                printf("C%d - %d ", i+1, e.n);
            }
        }
    }

    //Si ya no hay mas clientes que atender
    if(atendidos>=100){
        aux = 0;
        for(i=0;i<cajas;i++){
            if (!Empty(&cajeras[i])){
                aux++;
            }
        }
        if(aux==0){
            CerrarTienda(tienda, cajas, atendidos, atendidosC, tiempo);
            break;
        }
    }

    //Si el tiempo es multiplo del de llegada de los clientes
    if(tiempo % clienteT == 0){
        clienteN++; //Incrementar el numero de clientes
        filaC=rand() %cajas; //Escoger la fila para formarse aleatoriamente

        //Mantener el control del tiempo de cuando una caja empezo a tener clientes
        if(Empty(&cajeras[filaC])){
            tiempoC[filaC] = (cajasT[filaC] - (tiempo % cajasT[filaC])) % cajasT[filaC];
        }

        //Agregar Cliente a la Caja
        AgregarCliente(&cajeras[filaC], cajas, clienteN, filaC);
    }
}
return;
}

/*
void AgregarCliente(cola *cajera, int clienteN, int filaC)
Recibe: cola * Referencia/Dirección a la cola de un cajero, int Numero de clientes, int Numero de Caja
Devuelve: void (No retorna valor explicito)
Observaciones: Función que agrega un cliente a la cola de un cajero, mostrando tambien como este es
                ingresado a la cola en pantalla.
*/
void AgregarCliente(cola *cajera, int cajas, int clienteN, int filaC){
    int espacio=5+(7*filaC)+((cajas==10)?0:((cajas%2==0)?0:4))+((10-cajas)/2)*7;
    int fila=10, columna=espacio;
    elemento e;
    int tam;

    if(Empty(cajera)){
        MoverCursor(columna,fila);
        if(clienteN<100){
            printf(" %d", clienteN);
        } else if (clienteN<10000){
            printf(" %d", clienteN);
        } else {
            printf("%d", clienteN);
        }
    } else {
        tam = Size(cajera);
        fila++;
        if(tam<10){
            MoverCursor(columna,fila+tam);
            if(clienteN<100){
                printf(" %d", clienteN);
            } else if (clienteN<10000){
                printf(" %d", clienteN);
            } else {
                printf("%d", clienteN);
            }
        }
    }
}

```

```

        } else {
            tam-=9;
            MoverCursor(columna, fila+10);
            if(tam<10){
                printf("  +%d", tam);
            } else if (tam<1000){
                printf(" +%d", tam);
            } else {
                printf("+%d", tam);
            }
        }
    }

    EsperarMiliSeg(20);
    e.n = clienteN;
    Queue(cajera, e);
}

/*
    elemento QuitarCliente(cola *cajera, int filaC)
    Recibe:   cola * Referencia/Dirección a la cola de un cajero, int Numero de Caja
    Devuelve: elemento Elemento
    Observaciones:   Función que quita un cliente de cola de un cajero, mostrando tambien como este es
                    eliminado de la cola en pantalla, retornando el valor del elemento.
*/
elemento QuitarCliente(cola *cajera, int cajas, int filaC){
    int espacio=5+(7*filaC)+(cajas==10)?0:((cajas%2==0)?0:4)+((10-cajas)/2)*7);
    int fila=10, columna=espacio;
    int tam, i;
    elemento e, aux;

    e = Dequeue(cajera);

    MoverCursor(columna, fila);
    if(Empty(cajera)){
        printf("      ");
    } else {
        tam = Size(cajera);

        for(i=0; i<=tam+1; i++){
            MoverCursor(columna, fila+i);
            printf("      ");
            MoverCursor(columna, fila+i);

            if(i==tam+1)break;
            if(i==11)break;

            aux = Element(cajera, i==0?1:i);
            if(aux.n<100){
                printf("  %d", aux.n);
            } else if (aux.n<10000){
                printf(" %d", aux.n);
            } else {
                printf("%d", aux.n);
            }

            if(i==0)i++;
        }

        if(tam>10){
            tam-=10;
            if(tam<10){
                printf("  +%d", tam);
            } else if (tam<1000){
                printf(" +%d", tam);
            } else {
                printf("+%d", tam);
            }
        }
    }

    EsperarMiliSeg(10);
    return e;
}

/*
    void AbrirTienda(char tienda[], int cajas)
    Recibe:   char Cadena del nombre de la tienda, int Numero de cajas
    Devuelve: void (No retorna valor explicito)
    Observaciones:   Función que abre la tienda de manera animada en pantalla.
*/
void AbrirTienda(char tienda[], int cajas){
    DibujaPresentacion();
    DibujaMarco(tienda);
    DibujaCajas(cajas);
    DibujaEstantes(cajas);
    DibujaAnuncioAbrir();
}

/*
    void CerrarTienda(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo)
    Recibe:   char Cadena del nombre de la tienda, int Numero de cajas, int Numero de clientes atendidos,
            int Arreglo de clientes atendidos por caja, int Tiempo de ejecución
    Devuelve: void (No retorna valor explicito)
    Observaciones:   Función que cierra la tienda de manera animada en pantalla.
*/

```

```

void CerrarTienda(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo){
    DibujaAnuncioCerrar();
    EsperarMiliSeg(600);
    DibujaCortina();
    EsperarMiliSeg(1200);
    DibujaCartel();
    DibujaCartelDatos(tienda, cajas, atendidos, atendidosC, tiempo);
}

/*
    void DibujaPresentacion()
    Recibe: void (No recibe valor explicito)
    Devuelve: void (No retorna valor explicito)
    Observaciones: Función que da una presentación a la simulación 01.
*/
void DibujaPresentacion(){
    int columna, fila, i;

    BorrarPantalla();

    //Crear Particulas Aleatorias
    srand(time(NULL));
    EsperarMiliSeg(2200);
    for (i=1;i<=500;i++) {
        columna=(rand()%(ANCHO-1))+1;
        fila=(rand()%(ALTO-1))+1;
        MoverCursor(columna,fila);
        printf("%*");
        if(i%100 == 0){
            EsperarMiliSeg(800);
        }
    }

    //Crear Primer Cuadro
    EsperarMiliSeg(2200);
    for(columna=22;columna<=57;columna++){
        for(fila=4;fila<=13;fila++){
            MoverCursor(columna,fila);
            if((columna>22 && columna<57) && (fila==4 || fila==13)){
                printf(" ");
            } else if ((fila>4 && fila<=13) && (columna==22 || columna==57)){
                printf("|");
            } else {
                printf(" ");
            }
        }
    }

    //Poner datos
    EsperarMiliSeg(1800);
    MoverCursor(28,6);
    printf("Erendil Aguilar Avendaño");
    MoverCursor(28,9);
    printf("Iker Itzae Aguilar Souza");
    MoverCursor(35,12);
    printf("PRESENTAN");

    //Crear Segundo Cuadro
    EsperarMiliSeg(2200);
    for(columna=22;columna<=57;columna++){
        for(fila=17;fila<=20;fila++){
            MoverCursor(columna,fila);
            if((columna>22 && columna<57) && (fila==17 || fila==20)){
                printf(" ");
            } else if ((fila>17 && fila<=20) && (columna==22 || columna==57)){
                printf("|");
            } else {
                printf(" ");
            }
        }
    }

    //Poner datos
    EsperarMiliSeg(1800);
    MoverCursor(34,19);
    printf("SIMULACION 01");

    EsperarMiliSeg(5000);
    BorrarPantalla();
}

/*
    void DibujaMarco(char tienda[])
    Recibe: char Cadena del nombre de la tienda
    Devuelve: void (No retorna valor explicito)
    Observaciones: Función que crea el todo el marco del Supermercado asi como sus datos iniciales.
*/
void DibujaMarco(char tienda[]){
    int columna,fila,i;
    int Esperar = 5;

    //Crear columnas
    for(columna=1,fila=2;fila<ALTO;fila++){
        //Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
        MoverCursor(columna,fila);

```



```

        printf("|");
        MoverCursor(ANCHO-1, fila);
        printf("|");
        EsperarMiliSeg(Esperar);
    }

    //Crear filas
    for(columna=2, fila=1; columna<ANCHO-1; columna++){
        //Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
        MoverCursor(columna, fila);
        printf("_");
        MoverCursor(columna, fila+4);
        printf("_");
        MoverCursor(columna, ALTO-1);
        printf(" ");
        EsperarMiliSeg(Esperar);
    }

    //Crear texto
    MoverCursor(5,3);
    printf("Tienda:\n%s\n", tienda);
    EsperarMiliSeg(200);
    MoverCursor(5,4);
    printf("Num. Clientes Atendidos: 0");
    EsperarMiliSeg(200);
    MoverCursor(42,4);
    printf("Ultimo movimiento:");
    EsperarMiliSeg(200);
    MoverCursor(65,3);
    printf("[[      ]]");
    EsperarMiliSeg(Esperar);
}

/*
void DibujaCajas(int cajas)
Recibe:  int Numero de cajas
Devuelve: void (No retorna valor explicito)
Observaciones:  Función que crea las cajas que usara el Supermercado con base en un
                valor de cajas dado.
*/
void DibujaCajas(int cajas){
    int espacio=4+((cajas==10)?0:((cajas%2==0)?0:4)+((10-cajas)/2)*7);
    int columna=espacio, fila=6, i, j;
    int Esperar = 5;

    //Recorrer cada columna
    for(columna=espacio, i=1; i<=cajas; columna+=7, i++){
        //Crear Caja
        for(j=1; j<=6; j++){
            MoverCursor(columna+j, fila);
            printf("_");
            MoverCursor(columna+j, fila+3);
            printf(" ");
            EsperarMiliSeg(Esperar);
        }

        //Crear Lineas
        for(j=1; j<=15; j++){
            if(j==4){
                MoverCursor(columna, fila+j);
                if(i==1){
                    printf(":");
                } else {
                    printf("-");
                }
                MoverCursor(columna+7, fila+j);
                if(i==cajas){
                    printf(":");
                } else {
                    printf("-");
                }
                continue;
            }
            if(j==5){
                MoverCursor(columna, fila+j);
                if(i==1){
                    printf(":");
                } else {
                    printf(".");
                }
                MoverCursor(columna+7, fila+j);
                if(i==cajas){
                    printf(":");
                } else {
                    printf(".");
                }
                continue;
            }
            MoverCursor(columna, fila+j);
            printf(" ");
            MoverCursor(columna+7, fila+j);
            printf(" ");
            EsperarMiliSeg(Esperar);
        }
        MoverCursor(columna+(i==10?2:3), fila+2);
    }
}

```

```

        printf("C%d",i);
    }
}

/*
void DibujaEstantes(int cajas)
Recibe: int Numero de cajas
Devuelve: void (No retorna valor explicito)
Observaciones: Función que dibuja Estantes de decoración para la simulación.
*/
void DibujaEstantes(int cajas){
    int espacio=3+(7*cajas)+((cajas==10)?0:((cajas%2==0)?0:4)+((10-cajas)/2)*7);
    int columna=4,fila=6,i,j,k;
    int Esperar = 5;
    int conteo = 1;

    /**Comprobar que hay menos de 10 cajas**/
    //if(cajas==10) return;

    /**Crear Estantes Grandes**/
    for(columna=4,fila=6,i=1;i<=2;columna+=espacio,i++){
        for(j=0;j<5;j++){
            if(j==0){
                MoverCursor(columna+1,fila+j);
                for(k=0;k<(10-cajas)*3-2;k++){
                    printf("_");
                }
            } else {
                MoverCursor(columna,fila+j);
                if(j%2!=0){
                    for(k=0;k<10-cajas;k++){
                        if(k%2==0) printf("|/|");
                        else printf("\\\\|");
                    }
                } else {
                    for(k=0;k<10-cajas;k++){
                        if(k%2==0) printf("\\\\|");
                        else printf("|/|");
                    }
                }
            }
            EsperarMiliSeg(Esperar);
        }
    }

    /**Crear Estantes Chicos**/
    for(columna=4,fila=12,i=1;i<=2;columna+=espacio,i++){
        for(j=0;j<5;j++){
            MoverCursor(columna,fila+j*2);
            for(k=0;k<10-cajas;k++){
                printf("#");
            }
            EsperarMiliSeg(Esperar);
        }
    }
}

/*
void DibujaCortina()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea una cortina desendente en la pantalla.
*/
void DibujaCortina(){
    int columna,fila,i,j;

    EsperarMiliSeg(800);

    for(fila=1,columna=1;fila<ALTO;fila++){
        MoverCursor(columna,fila);
        printf("|||");
        MoverCursor(ANCHO-4,fila);
        printf("|||");

        if(fila-1>0){
            MoverCursor(columna,fila-1);
            printf(" | |");
            for(i=0;i<14;i++){
                printf(" _ |");
            }
            printf(" | |");
        }

        if(fila-2>0){
            MoverCursor(columna,fila-2);
            printf(" | |");
            for(i=0;i<15;i++){
                printf(" _ |");
            }
        }

        if(fila-3>0){
            MoverCursor(columna,fila-3);
            printf(" | |");
            for(i=0;i<15;i++){
                printf(" _ |");
            }
        }
    }
}

```

```

        EsperarMiliSeg(200);
    }
    MoverCursor(4,ALTO-1);
    for(i=0;i<14;i++){
        printf("      ");
    }
    printf(" ");
    EsperarMiliSeg(1500);
}

/*
void DibujaCartel()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea un cartel desendente en la pantalla.
*/
void DibujaCartel(){
    int columna,fila,i,j;

    for(fila=1,columna=9;fila<ALTO-4;fila++){
        MoverCursor(columna,fila);
        printf("|");
        for(i=0;i<59;i++){
            printf("_");
        }
        printf("|");

        if(fila-1>0){
            MoverCursor(columna,fila-1);
            printf("|");
            for(i=0;i<59;i++){
                printf(" ");
            }
            printf("|");
        }

        if(fila-14>0){
            MoverCursor(columna,fila-14);
            printf(" ");
            for(i=0;i<59;i++){
                printf("_");
            }
            printf(" ");
        }

        if(fila-15>0){
            MoverCursor(columna,fila-15);
            for(i=0;i<12;i++){
                if(i==2 || i==10)
                    printf("||  |");
                else
                    printf(" |  |");
            }
        }

        EsperarMiliSeg(200);
    }
}

/*
void DibujaCartelDatos(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo)
Recibe: char Cadena del nombre de la tienda, int Numero de cajas, int Numero de clientes atendidos,
        int Arreglo de clientes atendidos por caja, int Tiempo de ejecución
Devuelve: void (No retorna valor explicito)
Observaciones: Función que agrega los datos al cartel del finalizamiento de la simulación.
*/
void DibujaCartelDatos(char tienda[], int cajas, int atendidos, int atendidosC[], int tiempo){
    int columna,fila,i,j,k;
    int mejorC = 0;

    //Calcular mejor Caja
    for(i=0;i<cajas;i++){
        if(mejorC < atendidosC[i]){
            mejorC = atendidosC[i];
        }
    }

    //Mostrar datos finales
    for(columna=12,fila=7,i=1;i<=5;fila+=2,i++){
        EsperarMiliSeg(600);
        MoverCursor(columna,fila);

        if(i==1){
            printf("\n%s\n", tienda);
            columna--;
        }

        if(i==2)
            printf("Tiempo abierta:      %d Milisegundos", tiempo);

        if(i==3)
            printf("Clientes Atendidos:      %d Clientes", atendidos);

        if(i==4){
            printf("Cajas mas eficientes:      ");
            for(j=0,k=0;j<cajas;j++){
                if(mejorC == atendidosC[j]){

```

```

        if(k>0)
            printf(",");
        printf("C%d", j+1);
        k++;
    }
}

if(i==5){
    printf("Clientes Atendidos por Caja:");
    for(columna+=2, fila++, j=0; j<cajas; j++){
        EsperarMiliSeg(200);
        MoverCursor(columna, fila);
        printf("C%d: %d", j+1, atendidosC[j]);
        if((j+1)%3 != 0){
            fila++;
        } else {
            fila-=2;
            columna+=14;
        }
    }
}

}

/*
void DibujaAnuncioAbrir()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que da un anuncio en pantalla de que la tienda va a abrir.
*/
void DibujaAnuncioAbrir(){
    int columna=67, fila=3, i;
    char anuncio[] = "ABIERTO";
    int Esperar = 800;

    //Escribir Conteo
    MoverCursor(columna, fila);
    for(i=3; i>0; i--){
        EsperarMiliSeg(Esperar);
        printf(" %d", i);
    }
    EsperarMiliSeg(Esperar);

    //Quitar Conteo
    MoverCursor(columna, fila);
    printf(" ");
    EsperarMiliSeg(Esperar);

    //Escribir ABIERTO Lentamente
    MoverCursor(columna, fila);
    for(i=0; i<7; i++){
        EsperarMiliSeg(Esperar-300);
        printf("%c", anuncio[i]);
    }

    //Parpadeo de Anuncio
    for(i=0; i<4; i++){
        MoverCursor(columna, fila);
        EsperarMiliSeg(Esperar-200);
        if(i%2 == 0){
            printf(" ");
        } else {
            printf("%s", anuncio);
        }
    }
}

/*
void DibujaAnuncioCerrar()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que da un anuncio en pantalla de que la tienda va a cerrar.
*/
void DibujaAnuncioCerrar(){
    int columna=67, fila=3, i;
    char anuncio[] = "CERRADO";
    int Esperar = 800;

    //Quitar Cartel de Abierto
    MoverCursor(columna, fila);
    printf(" ");
    EsperarMiliSeg(Esperar);

    //Escribir Cerrado Lentamente
    MoverCursor(columna, fila);
    for(i=0; i<7; i++){
        EsperarMiliSeg(Esperar-300);
        printf("%c", anuncio[i]);
    }

    //Parpadeo de Anuncio
    for(i=0; i<8; i++){
        MoverCursor(columna, fila);
        EsperarMiliSeg(Esperar-200);

```

```
        if(i%2 == 0){
            printf("      ");
        } else {
            printf("%s", anuncio);
        }
    }
}
```

Simulación 02: Ejecución de procesos en el sistema operativo.

```
/*
Autores:  Erendil Aguilar Avendaño
          Iker Itzae Aguilar Souza
Versión 1.4 (06 de Junio 2024)
Grupo:    2CM2
Materia:  Algoritmos y Estructuras de Datos
Práctica 04: Simulaciones con el TAD Cola

=== Simulaciones con el TAD Cola ===

Descripción: Programa que con el uso de colas simula el comportamiento de las Ejecuciones de Procesos en un Sistema
Operativo de manera animada.

Observaciones: El programa requiera de la libreria "presentacion.h", la cuál tiene las implementaciones
para mover el cursor de la pantalla, esperar un tiempo y borrar pantalla, así como las librerías "TADColaEst.h"
y "TADCola/TADColaDin.h" para el uso correcto de las colas, la compilación debiera incluir las definiciones de
las funciones según la plataforma que se este utilizando (Windows o Linux).

Compilación: gcc -o simulacion_02 simulacion_02.c presentacion/presentacion(Win|Lin).o
TADCola/TADCola(Din|Est|EstCirc).o (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
objeto"
gcc -o simulacion_02 simulacion_02.c presentacion/presentacion(Win|Lin).c
TADCola/TADCola(Din|Est|EstCirc).c (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
fuente"
Ejecución: Windows simulacion_02.exe & Linux ./simulacion_02
*/

//LIBRERIAS
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include <time.h>
#include <math.h>
#include <locale.h>
#include "presentacion/presentacion.h"
//#include "TADCola/TADColaEst.h" //Si se usa la implemtentación estática (TADColaEst.c|TADColaEstCirc.c)
#include "TADCola/TADColaDin.h" //Si se usa la implemtentación dinámica (TADColaDin.c)

//CONSTANTES
#define ALTO 24 //Se piensa en un pantalla de 24 filas x 79 columnas
#define ANCHO 79
#define TIEMPO_BASE 1000 //Tiempo base en milisegundos

//FUNCIONES
void PedirRequisitos(cola *Faltantes, int *procesos);

void Simulacion(cola *Faltantes, int procesos);

void AgregarProceso(cola *Ejecucion, cola *Faltantes);
void QuitarProceso(cola *Faltantes);
void TerminarProceso(cola *Finalizados, cola *Faltantes);

void AbrirSistemaOperativo(cola *Faltantes);
void CerrarSistemaOperativo(cola *Finalizados);

void DibujaPresentacion();
void DibujaMarco();
void DibujaCuadros();
void DibujaElementoAgregar(elemento e, int tipo);
void DibujaElementoEliminar(int tipo);
void DibujaCortina();
void DibujaCartel();
void DibujaCartelDatos(cola *Finalizados);
void DibujaAnuncioAbrir();
void DibujaAnuncioCerrar();

//PROGRAMA PRINCIPAL
int main(void)
{
    cola Faltantes;
    int procesos;

    setlocale(LC_ALL, "");

    //INICIALIZAR COLA FALTANTES
    Initialize(&Faltantes);

    /***PEDIR REQUISITOS***/
    PedirRequisitos(&Faltantes, &procesos);

    /***INCIAR LA TIENDA***/
    Simulacion(&Faltantes, procesos);

    MoverCursor(ANCHO,ALTO);
}

/*
void PedirRequisitos(cola *Faltantes, int *procesos)
Recibe: cola * Referencia/Dirección a la cola de procesos Faltantes, int * Referencia/Dirección al número de
procesos
*/
```

```

Devuelve: void (No retorna valor explicito)
Observaciones: Función que pide los datos como le numero de procesos a ejecutar, con su respectivo, Nombre,
Nombre de Actividad, Segundos para completar la Ejecución y un ID
Generado automaticamente.
*/
void PedirRequisitos(cola *Faltantes, int *procesos){
    char Nombre[32], Actividad[52], ID[8], correcto, ch1, ch2, c;
    int Aprocesos, segundos, num1, num2, i, j, k;
    elemento e, aux;

    BorrarPantalla();

    /**Pedir el numero de procesos***/
    printf("¿Cuántos procesos quiere que se ejecuten en el sistema operativo?\n");
    do{
        scanf("%d", &Aprocesos);
        while ((c = getchar()) != '\n' && c != EOF);
        if(Aprocesos<1 || Aprocesos>100){
            printf("Numero de procesos no valido (0<N<101).\n");
        }
    }while(Aprocesos<1 || Aprocesos>100);

    for(i=0;i<Aprocesos;i++){
        /**Pedir el Nombre del Proceso***/
        correcto=0;
        printf("¿Como se llamara el proceso %d?\n", i+1);
        do{
            fgets(Nombre, sizeof(Nombre), stdin);
            // Verificar si la longitud de la cadena ingresada excede 30 caracteres (sin contar el '\n'
de fgets)
            if (strlen(Nombre) == 31 && Nombre[30] != '\n') {
                printf("El nombre del proceso supera los 30 caracteres.\n");
                while (getchar() != '\n');
                //exit(1);
            } else {
                //En caso de un salto de linea
                if (Nombre[strlen(Nombre) - 1] == '\n') {
                    Nombre[strlen(Nombre) - 1] = '\0';
                }
                //Quitar espacios innecesarios en el nombre
                for(j=0;j<strlen(Nombre);j++){
                    if(Nombre[j] == ' '){
                        while(Nombre[j+1] == ' '){
                            for(k=j+1;k<strlen(Nombre);k++){
                                Nombre[k] = Nombre[k+1];
                            }
                        }
                        while(Nombre[j] == ' ' && (j==0 || j==strlen(Nombre)-1)){
                            for(k=j;k<strlen(Nombre);k++){
                                Nombre[k] = Nombre[k+1];
                            }
                        }
                    }
                }
                if(strlen(Nombre)!=0){
                    correcto = 1;
                } else {
                    printf("El nombre del proceso no es valido.\n");
                }
            }
        }while(correcto == 0);

        /**Pedir la Actividad del Proceso***/
        correcto=0;
        printf("¿Cual es la actividad el proceso %d?\n", i+1);
        do{
            fgets(Actividad, sizeof(Actividad), stdin);
            // Verificar si la longitud de la cadena ingresada excede 30 caracteres (sin contar el '\n'
de fgets)
            if (strlen(Actividad) == 51 && Actividad[50] != '\n') {
                printf("El nombre de la actividad del proceso supera los 50 caracteres.\n");
                while (getchar() != '\n');
                //exit(1);
            } else {
                //En caso de un salto de linea
                if (Actividad[strlen(Actividad) - 1] == '\n') {
                    Actividad[strlen(Actividad) - 1] = '\0';
                }
                //Quitar espacios innecesarios en la Actividad
                for(j=0;j<strlen(Actividad);j++){
                    if(Actividad[j] == ' '){
                        while(Actividad[j+1] == ' '){
                            for(k=j+1;k<strlen(Actividad);k++){
                                Actividad[k] = Actividad[k+1];
                            }
                        }
                        while(Actividad[j] == ' ' && (j==0 || j==strlen(Actividad)-1)){
                            for(k=j;k<strlen(Actividad);k++){
                                Actividad[k] = Actividad[k+1];
                            }
                        }
                    }
                }
                if(strlen(Actividad)!=0){

```

```

        } else {
            correcto = 1;
            printf("El nombre del proceso no es valido.\n");
        }
        correcto = 1;
    }
}while(correcto == 0);

/**Crear ID del proceso**/
num1=i+1;
num2=1;
k=1;
if(!Empty(Faltantes)){
    for(j=Size(Faltantes);j>0;j--){
        aux = Element(Faltantes, j);
        if(strcmp(Nombre, aux.Nom) == 0){
            ch1 = aux.ID[3];
            ch2 = aux.ID[4];
            num2 = atoi(&aux.ID[5])+1;
            k=0;
            break;
        }
    }
}
if(k){
    ch1=Nombre[0];
    ch2='0';
    for(j=0;j<strlen(Nombre);j++){
        if(Nombre[j] == ' ' && Nombre[j+1] != '\0'){
            ch2=Nombre[j+1];
        }
    }
    if(ch1>='a' && ch1<='z') ch1+='A'-'a';
    if(ch2>='a' && ch2<='z') ch2+='A'-'a';
}
snprintf(ID, sizeof(ID), "%03d%c%c%02d", num1, ch1, ch2, num2);

/**Pedir el tiempo de Proceso**/
printf("¿Cual es el tiempo de proceso para el proceso %d?\n", i+1);
do{
    scanf("%d", &segundos);
    while ((c = getchar()) != '\n' && c != EOF);
    if(segundos<1 || segundos>3600){
        printf("Numero de procesos no valido (0<N<3601).\n");
    }
}while(segundos<1 || segundos>3600);

/**Agregar Proceso a la Cola**/
strncpy(e.Nom, Nombre, 31);
strncpy(e.Act, Actividad, 51);
strncpy(e.ID, ID, 8);
e.seg = segundos;
e.temF = segundos;
e.temT = 0;

Queue(Faltantes, e);
printf("\n");
}

*procesos = Aprocesos;
}

/*
void Simulacion(cola *Faltantes, int procesos)
Recibe: cola * Referencia/Dirección a la cola de procesos Faltantes, int Número de procesos
Devuelve: void (No retorna valor explicito)
Observaciones: Función que simulara el comportamiento de ejecución de un sistema operativo, con base en un
número determinado de procesos, ejecutando cada proceso poco a poco por
segundo, para ejecutar
de manera equitativa todos los procesos.
*/
void Simulacion(cola *Faltantes, int procesos){
    int tiempo = 0, clienteN = 0, atendidos = 0;
    int i, filaC, aux;
    elemento e;

    //Inicializar la función Rand
    srand(time(NULL));

    //Crear las colas
    cola Ejecucion, Finalizados;

    //Inicializar colas
    Initialize(&Ejecucion);
    Initialize(&Finalizados);

    //Iniciar Tienda
    AbrirSistemaOperativo(Faltantes);

    //Ciclo infinito de la simulación
    while(1){
        tiempo++; //Incrementar el contador de tiempo
    }
}

```



```

        //Checar estado del proceso
        if(!Empty(&Ejecucion)){
            e = Element(&Ejecucion, 1);
            if(e.temF == 0){
                //Si el proceso ya acabo
                Queue(&Finalizados, Dequeue(&Ejecucion));
                TerminarProceso(&Finalizados, Faltantes);
            } else {
                //Si el proceso no a acabado
                Queue(Faltantes, Dequeue(&Ejecucion));
                QuitarProceso(Faltantes);
            }
        }

        //Si hay Procesos por hacer
        if (!Empty(Faltantes)){
            e = Dequeue(Faltantes);
            e.temT=tiempo;
            e.temF--;
            //Agregar Ejecución
            Queue(&Ejecucion, e);
            AgregarProceso(&Ejecucion, Faltantes);
        }

        EsperarMiliSeg(TIEMPO_BASE); //Esperar el tiempo base (1 segundo)

        if(Empty(Faltantes) && Empty(&Ejecucion)){
            //Terminar sistema operativo
            CerrarSistemaOperativo(&Finalizados);
            break;
        }
    }
    return;
}

/*
void AgregarProceso(cola *Ejecucion, cola *Faltantes)
Recibe: cola * Referencia/Dirección a la cola de procesos en Ejecución, cola * Referencia/Dirección a la cola
de procesos Faltantes
Devuelve: void (No retorna valor explicito)
Observaciones: Función que genera de manera animada el cambio de un proceso Faltante, para el apartado de
Ejecución.
*/
void AgregarProceso(cola *Ejecucion, cola *Faltantes){
    int columna, fila, i;
    int Esperar = 5;
    elemento e;

    DibujaElementoEliminar(2);
    if(!Empty(Faltantes)){
        DibujaElementoAgregar(Element(Faltantes, 1), 2);
    } else {
        e = Element(Ejecucion, 1);
        if(e.temF != 0){
            DibujaElementoAgregar(Element(Ejecucion, 1), 2);
            EsperarMiliSeg(100);
            DibujaElementoEliminar(2);
        }
    }
    EsperarMiliSeg(Esperar);

    //Eliminar Flecha
    for(columna=18,fila=10,i=0;i<9;fila++,i++){
        MoverCursor(columna,fila);
        printf(" ");
    }
    for(columna++,fila--,i=0;i<12;columna++,i++){
        MoverCursor(columna,fila);
        printf(" ");
    }

    //Animación de Agregar Flecha
    for(columna=30,fila=18,i=0;i<12;columna--,i++){
        MoverCursor(columna,fila);
        printf("-");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna,fila);
    printf("\n");
    EsperarMiliSeg(Esperar);
    for(fila--,i=0;i<7;fila--,i++){
        MoverCursor(columna,fila);
        printf("|");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna,fila);
    printf("^");
    EsperarMiliSeg(Esperar);

    DibujaElementoAgregar(Element(Ejecucion, 1), 0);
    EsperarMiliSeg(Esperar);
}

```

```

/*
    void QuitarProceso(cola *Faltantes)
    Recibe: cola * Referencia/Dirección a la cola de procesos Faltantes.
    Devuelve: void (No retorna valor explicito)
    Observaciones: Función que genera de manera animada el cambio de un proceso en Ejecución, para el apartado
    de procesos Faltantes
*/
void QuitarProceso(cola *Faltantes){
    int columna, fila, i;
    int Esperar = 5;

    DibujaElementoEliminar(0);
    EsperarMiliSeg(Esperar);

    //Eliminar Flecha
    for(columna=25, fila=10, i=0; i<5; fila++, i++){
        MoverCursor(columna, fila);
        printf(" ");
    }
    for(columna++, fila--, i=0; i<5; columna++, i++){
        MoverCursor(columna, fila);
        printf(" ");
    }

    //Animación de Agregar Flecha
    for(columna=25, fila=10, i=0; i<4; fila++, i++){
        MoverCursor(columna, fila);
        printf("|");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna, fila);
    printf("\n");
    EsperarMiliSeg(Esperar);
    for(columna++, i=0; i<4; columna++, i++){
        MoverCursor(columna, fila);
        printf("-");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna, fila);
    printf(">");
    EsperarMiliSeg(Esperar);

    DibujaElementoEliminar(1);
    DibujaElementoAgregar(Element(Faltantes, Size(Faltantes)), 1);
    EsperarMiliSeg(Esperar);
}

/*
    void TerminarProceso(cola *Finalizados, cola *Faltantes)
    Recibe: cola * Referencia/Dirección a la cola de procesos Finalizados, cola * Referencia/Dirección a la cola de
    procesos Faltantes
    Devuelve: void (No retorna valor explicito)
    Observaciones: Función que genera de manera animada el cambio de un proceso en Ejecución, para el apartado
    de procesos Finalizados.
*/
void TerminarProceso(cola *Finalizados, cola *Faltantes){
    int columna, fila, i;
    int Esperar = 5;

    DibujaElementoEliminar(0);
    if(Empty(Faltantes)){
        DibujaElementoEliminar(1);
    }
    EsperarMiliSeg(Esperar);

    //Eliminar Flecha
    for(columna=11, fila=10, i=0; i<13; fila++, i++){
        MoverCursor(columna, fila);
        printf(" ");
    }
    for(columna++, fila--, i=0; i<19; columna++, i++){
        MoverCursor(columna, fila);
        printf(" ");
    }

    //Animación de Agregar Flecha
    for(columna=11, fila=10, i=0; i<12; fila++, i++){
        MoverCursor(columna, fila);
        printf("|");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna, fila);
    printf("\n");
    EsperarMiliSeg(Esperar);
    for(columna++, i=0; i<18; columna++, i++){
        MoverCursor(columna, fila);
        printf("-");
        EsperarMiliSeg(Esperar);
    }
    MoverCursor(columna, fila);
    printf(">");
    EsperarMiliSeg(Esperar);
}

```

```

        DibujaElementoEliminar(3);
        DibujaElementoAgregar(Element(Finalizados, Size(Finalizados)), 3);
        EsperarMiliSeg(Esperar);
    }

    /*
        void AbrirSistemaOperativo(cola *Faltantes)
        Recibe: cola * Referencia/Dirección a la cola de procesos Faltantes.
        Devuelve: void (No retorna valor explicito)
        Observaciones: Función que abre el Sistema Operativo de manera animada en pantalla.
    */
    void AbrirSistemaOperativo(cola *Faltantes){
        DibujaPresentacion();
        DibujaMarco();
        DibujaCuadros();
        DibujaAnuncioAbrir();
        DibujaElementoAgregar(Element(Faltantes, 1), 2);
    }

    /*
        void CerrarSistemaOperativo(cola *Finalizados)
        Recibe: cola * Referencia/Dirección a la cola de procesos Finalizados
        Devuelve: void (No retorna valor explicito)
        Observaciones: Función que cierra el Sistema Operativo de manera animada en pantalla.
    */
    void CerrarSistemaOperativo(cola *Finalizados){
        DibujaAnuncioCerrar();
        EsperarMiliSeg(600);
        DibujaCortina();
        EsperarMiliSeg(1200);
        DibujaCartel();
        DibujaCartelDatos(Finalizados);
    }

    /*
        void DibujaPresentacion()
        Recibe: void (No recibe valor explicito)
        Devuelve: void (No retorna valor explicito)
        Observaciones: Función que da una presentación a la simulación 02.
    */
    void DibujaPresentacion(){
        int columna, fila, i;

        BorrarPantalla();

        //Crear Particulas Aleatorias
        srand(time(NULL));
        EsperarMiliSeg(2200);
        for (i=1; i<=500; i++) {
            columna=(rand()%(ANCHO-1))+1;
            fila=(rand()%(ALTO-1))+1;
            MoverCursor(columna, fila);
            printf("x");
            if(i%100 == 0){
                EsperarMiliSeg(800);
            }
        }

        //Crear Primer Cuadro
        EsperarMiliSeg(2200);
        for(columna=22; columna<=57; columna++){
            for(fila=4; fila<=13; fila++){
                MoverCursor(columna, fila);
                if((columna>22 && columna<57) && (fila==4 || fila==13)){
                    printf(" ");
                } else if ((fila>4 && fila<=13) && (columna==22 || columna==57)){
                    printf("|");
                } else {
                    printf(" ");
                }
            }
        }

        //Poner datos
        EsperarMiliSeg(1800);
        MoverCursor(28,6);
        printf("Erendil Aguilar Avendaño");
        MoverCursor(28,9);
        printf("Iker Itzae Aguilar Souza");
        MoverCursor(35,12);
        printf("PRESENTAN");

        //Crear Segundo Cuadro
        EsperarMiliSeg(2200);
        for(columna=22; columna<=57; columna++){
            for(fila=17; fila<=20; fila++){
                MoverCursor(columna, fila);
                if((columna>22 && columna<57) && (fila==17 || fila==20)){
                    printf(" ");
                } else if ((fila>17 && fila<=20) && (columna==22 || columna==57)){
                    printf("|");
                } else {
                    printf(" ");
                }
            }
        }
    }

```

```

    }

    //Poner datos
    EsperarMiliSeg(1800);
    MoverCursor(34,19);
    printf("SIMULACION 02");

    EsperarMiliSeg(5000);
    BorrarPantalla();
}

/*
void DibujaMarco()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea el todo el marco del Sistema Operativo asi como un Titulo.
*/
void DibujaMarco(){
    int columna, fila, i;
    int Esperar = 5;

    //Crear columnas
    for(columna=1, fila=2; fila<ALTO; fila++){
        //Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
        MoverCursor(columna, fila);
        printf("|");
        MoverCursor(ANCHO-1, fila);
        printf("|");
        EsperarMiliSeg(Esperar);
    }

    //Crear filas
    for(columna=2, fila=1; columna<ANCHO-1; columna++){
        //Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
        MoverCursor(columna, fila);
        printf("_");
        MoverCursor(columna, ALTO-1);
        printf("_");
        EsperarMiliSeg(Esperar);
    }

    //Crear titulo
    MoverCursor(24,3);
    printf("PROCESOS EN EL SISTEMA OPERATIVO");
    EsperarMiliSeg(200);
}

/*
void DibujaCuadros()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea los cuadros que usara el Sistema Operativo para mostrar el estado
de ciertos procesos.
*/
void DibujaCuadros(){
    int columna=8, fila=4, i, j;
    int Esperar = 5;

    /**Crear cuadro Principal**/
    //Crear lineas
    for(i=1; i<=62; i++){
        MoverCursor(columna+i, fila);
        printf(" ");
        MoverCursor(columna+i, fila+5);
        printf(" ");
        EsperarMiliSeg(Esperar);
    }

    //Crear columnas
    for(j=1; j<=5; j++){
        MoverCursor(columna, fila+j);
        printf("|");
        MoverCursor(columna+63, fila+j);
        printf("|");
        EsperarMiliSeg(Esperar);
    }

    /**Crear cuadros Secundarios**/
    //Crear cuadros
    for(columna=31, fila=12, i=1; i<=3; fila+=4, i++){
        //Crear lineas
        for(j=0; j<=2; j++){
            MoverCursor(columna+1+(j*41), fila);
            printf(" ");
            MoverCursor(columna+1+(j*41), fila+2);
            printf(" ");
            EsperarMiliSeg(Esperar);
        }

        //Crear lineas
        for(j=1; j<=2; j++){
            MoverCursor(columna, fila+j);
            printf("|");
            MoverCursor(columna+43, fila+j);
            printf("|");

```

```

        EsperarMiliSeg(Esperar);
    }
}

/**Agrega Titulos**/
MoverCursor(11,5);
printf("EJECUCIÓN");
EsperarMiliSeg(200);
MoverCursor(35,12);
printf("ANTERIOR");
EsperarMiliSeg(200);
MoverCursor(35,16);
printf("SIGUIENTE");
EsperarMiliSeg(200);
MoverCursor(35,20);
printf("FINALIZADO");
EsperarMiliSeg(200);
}

/*
void DibujaElementoAgregar(elemento e, int tipo)
Recibe: elemento Elemento, int tipo de dibujo
Devuelve: void (No retorna valor explicito)
Observaciones: Función que con base a la información de un elemento, y el tipo de dibujo, este
                agrega la información del elemento en el cuadro deseado.
*/
void DibujaElementoAgregar(elemento e, int tipo){
    int columna, fila;
    if(tipo==0){
        columna=14;fila=6;
        MoverCursor(columna,fila);
        printf("%s", e.Nom);
        MoverCursor(columna,++fila);
        printf("** %s", e.Act);
        MoverCursor(columna,++fila);
        printf("ID-%s", e.ID);
        MoverCursor(columna+21,fila);
        printf("TIEMPO TOTAL DE EJECUCION:%d", e.temT);
    } else {
        tipo--;
        columna=37;fila=13+(4*tipo);
        MoverCursor(columna,fila);
        printf("-%s-", e.Nom);
        MoverCursor(columna,++fila);
        printf("ID-%s", e.ID);
        MoverCursor(columna+12,fila);
        if(tipo!=2){
            printf("TIEMPO FALTANTE:%ds", e.temF);
        } else {
            printf("TIEMPO TARDADO:%ds", e.temT);
        }
    }
}

/*
void DibujaElementoEliminar(int tipo)
Recibe: int tipo de dibujo
Devuelve: void (No retorna valor explicito)
Observaciones: Función que con base al tipo de dibujo, este elimina la información contenida en
                el cuadro deseado.
*/
void DibujaElementoEliminar(int tipo){
    int columna, fila, i;
    if(tipo==0){
        columna=14;fila=6;
        for(i=0;i<3;i++){
            MoverCursor(columna,fila+i);
            printf("
        ");
        }
    } else {
        tipo--;
        columna=37;fila=13+(4*tipo);
        for(i=0;i<2;i++){
            MoverCursor(columna,fila+i);
            printf("
        ");
        }
    }
}

/*
void DibujaCortina()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea una cortina desendente en la pantalla.
*/
void DibujaCortina(){
    int columna,fila,i,j;

    EsperarMiliSeg(800);

    for(fila=1,columna=1;fila<ALTO;fila++){
        MoverCursor(columna,fila);
        printf("|||");
        MoverCursor(ANCHO-4,fila);
        printf("|||");
    }
}

```

```

        if(fila-1>0){
            MoverCursor(columna, fila-1);
            printf(" | |");
            for(i=0; i<14; i++){
                printf(" |__|");
            }
            printf(" | |");

        }

        if(fila-2>0){
            MoverCursor(columna, fila-2);
            printf(" | |");
            for(i=0; i<15; i++){
                printf("_ | |");
            }

        }

        if(fila-3>0){
            MoverCursor(columna, fila-3);
            printf(" | |");
            for(i=0; i<15; i++){
                printf(" | |");
            }

        }

        EsperarMiliSeg(200);
    }
    MoverCursor(4, ALTO-1);
    for(i=0; i<14; i++){
        printf(" ")
    }
    printf(" ");
    EsperarMiliSeg(1500);
}

/*
void DibujaCartel()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea un cartel desendente en la pantalla.
*/
void DibujaCartel(){
    int columna, fila, i, j;

    for(fila=1, columna=4; fila<ALTO-3; fila++){
        MoverCursor(columna, fila);
        printf(" |");
        for(i=0; i<69; i++){
            printf("_ ");
        }
        printf(" |");

        if(fila-1>0){
            MoverCursor(columna, fila-1);
            printf(" |");
            for(i=0; i<69; i++){
                printf(" ");
            }
            printf(" |");
        }

        if(fila-16>0){
            MoverCursor(columna, fila-16);
            printf(" ");
            for(i=0; i<69; i++){
                printf("_ ");
            }
            printf(" ");
        }

        if(fila-17>0){
            MoverCursor(columna, fila-17);
            for(i=0; i<14; i++){
                if(i==3 || i==11)
                    printf("|| |");
                else
                    printf(" | |");
            }

        }

        EsperarMiliSeg(200);
    }
}

/*
void DibujaCartelDatos(cola *Finalizados)
Recibe: cola * Referencia/Dirección a la cola de procesos Finalizados
Devuelve: void (No retorna valor explicito)
Observaciones: Función que agrega los datos al cartel del finalizamiento de la simulación.
*/
void DibujaCartelDatos(cola *Finalizados){
    int columna, fila, i, j, k;
    elemento e;

    //Mostrar datos finales
    for(columna=8, fila=6, i=1; i<=3; i++){
        EsperarMiliSeg(600);

        if(i==1){
            MoverCursor(columna+16, fila);

```

```

        printf("PROCESOS EN EL SISTEMA OPERATIVO");
        fila+=2;
    }

    if(i==2){
        MoverCursor(columna, fila);
        printf("Procesos mas rapidos:");
        MoverCursor(columna+48, fila);
        printf("Tiempo:");

        for(j=1; j<=Size(Finalizados)&& j<=3; j++){
            EsperarMiliSeg(200);
            MoverCursor(columna+3, fila+j);
            e = Element(Finalizados, j);
            printf(" ID-%s - %s ", e.ID, e.Nom);
            for(k=16+strlen(e.Nom); k<=49; k++){
                printf("-");
            }
            printf(" %ds", e.temT);
        }

        fila+=5;
    }

    if(i==3){
        if(Size(Finalizados)>3){
            MoverCursor(columna, fila);
            printf("Demas procesos:");
            for(columna+=3, fila++, j=4; j<=Size(Finalizados); j++){
                EsperarMiliSeg(200);
                MoverCursor(columna, fila);
                e = Element(Finalizados, j);

                if(j==21){
                    if(Size(Finalizados) == 21){
                        printf(" %s - %ds", e.ID, e.temT);
                    } else {
                        printf(" * +%d Procesos", Size(Finalizados) - 20);
                    }
                    break;
                }

                printf(" %s - %ds", e.ID, e.temT);
                if(j%3 != 0){
                    columna+=21;
                } else {
                    fila++;
                    columna=11;
                }
            }
        }
    }
}

/*
void DibujaAnuncioAbrir()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que da un anunció en pantalla de que el Sistema Operativo comenzará
con su funcionamiento.
*/
void DibujaAnuncioAbrir(){
    int columna=34, fila=7, i;
    int Esperar = 800;

    //Escribir Conteo
    for(i=3; i>0; i--){
        MoverCursor(columna, fila);
        EsperarMiliSeg(Esperar);
        printf("[ %d ]", i);
    }
    EsperarMiliSeg(Esperar);

    //Poner GO
    MoverCursor(columna, fila);
    printf("[ GO ]");
    EsperarMiliSeg(Esperar);

    //Quitar GO
    MoverCursor(columna, fila);
    printf(" ");
    EsperarMiliSeg(Esperar);
}

/*
void DibujaAnuncioCerrar()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que da un anunciado en pantalla de que el Sistema Operativo termino
con su funcionamiento.
*/
void DibujaAnuncioCerrar(){
    int columna=29, fila=7, i;
    char anuncio[] = "PROCESOS TERMINADOS";

```

```

int Esperar = 500;

//Escribir Anuncio Lentamente
MoverCursor(columna, fila);
for(i=0; i<19; i++){
    EsperarMiliSeg(Esperar-200);
    printf("%c", anuncio[i]);
}

//Parpadeo de Anuncio
for(i=0; i<6; i++){
    MoverCursor(columna, fila);
    EsperarMiliSeg(Esperar+100);
    if(i%2 == 0){
        printf("          ");
    } else {
        printf("%s", anuncio);
    }
}
}

```


Simulación 03: Banco

```
/*
Autores: Erendil Aguilar Avendaño
        Iker Itzae Aguilar Souza
Versión 1.5 (03 de Junio 2024)
Grupo: 2CM2
Materia: Algoritmos y Estructuras de Datos
Práctica 04: Simulaciones con el TAD Cola

=== Simulaciones con el TAD Cola ===

Descripción: Programa que con el uso de colas simula el comportamiento de un Banco de manera animada.

Observaciones: El programa requiera de la librería "presentacion.h", la cuál tiene las implementaciones
para mover el cursor de la pantalla, esperar un tiempo y borrar pantalla, así como las librerías "TADColaEst.h"
y "TADCola/TADColaDin.h" para el uso correcto de las colas, la compilación debiera incluir las definiciones de
las funciones según la plataforma que se este utilizando (Windows o Linux).

Compilación: gcc -o simulacion_03 simulacion_03.c presentacion/presentacion(Win|Lin).o
TADCola/TADCola(Din|Est|EstCirc).o (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
objeto"
gcc -o simulacion_03 simulacion_03.c presentacion/presentacion(Win|Lin).c
TADCola/TADCola(Din|Est|EstCirc).c (Win si se correra en Windows | Lin si se ejecutará en Linux) "Si se tiene el código
fuente"
Ejecución: Windows simulacion_03.exe & Linux ./simulacion_03
*/

//LIBRERIAS
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include <time.h>
#include <math.h>
#include <locale.h>
#include "presentacion/presentacion.h"
//#include "TADCola/TADColaEst.h" //Si se usa la implementación estática (TADColaEst.c|TADColaEstCirc.c)
#include "TADCola/TADColaDin.h" //Si se usa la implementación dinámica (TADColaDin.c)

//CONSTANTES
#define ALTO 24 //Se piensa en un pantalla de 24 filas x 79 columnas
#define ANCHO 79
#define TIEMPO_BASE 10 //Tiempo base en milisegundos

//FUNCIONES
void PedirRequisitos(int *cajeros, int *cajerosT, boolean *cajasB, int *clienteC, int *clienteU, int *clienteP);

void Simulacion(int cajeros, int cajerosT, boolean cajasB[], int clienteC, int clienteU, int clienteP);

void AgregarClienteCaja(boolean cajasB[], cola *cajero, cola *fila, int c, int f);
elemento QuitarClienteCaja(boolean cajasB[], cola *cajero, int c);
void AgregarClienteFila(cola *fila, int clienteN, int tipo);
elemento QuitarClienteFila(cola *fila, int tipo);

void AbrirBanco(int cajeros, boolean cajasB[]);

void DibujaPresentacion();
void DibujaMarco();
void DibujaCajas(int cajeros, boolean cajasB[]);
void DibujaFilas();
void DibujaEstantes();
void DibujaAnuncioAbrir();

//PROGRAMA PRINCIPAL
int main(void)
{
    int cajeros, cajerosT, clienteC, clienteU, clienteP;
    boolean cajasB[10];

    setlocale(LC_ALL, "");

    /***PEDIR REQUISITOS***/
    PedirRequisitos(&cajeros, &cajerosT, cajasB, &clienteC, &clienteU, &clienteP);

    /***INCIAR LA BANCO***/
    Simulacion(cajeros, cajerosT, cajasB, clienteC, clienteU, clienteP);

    MoverCursor(ANCHO,ALTO);
}

/*
PedirRequisitos(int *cajeros, int *cajerosT, boolean *cajasB, int *clienteC, int *clienteU, int *clienteP)
Recibe: int * Referencia/Dirección al número de cajeros, int * Referencia/Dirección al tiempo de atención de
los cajeros, boolean * Referencia/Dirección al arreglo de cajas siendo ocupadas, int *
Referencia/Dirección al tiempo de llegada de un Cliente, int * Referencia/Dirección al tiempo de llegada de un Usuario, int *
Referencia/Dirección al tiempo de llegada de un Preferente
Devuelve: void (No retorna valor explicito)
Observaciones: Función que pide los datos como el número de cajeros, tiempo de consulta de cajas, y el
tiempo
de llegada de los Clientes, Usuarios y Preferentes.
*/
```

```

*/
void PedirRequisitos(int *cajeros, int *cajerosT, boolean *cajasB, int *clienteC, int *clienteU, int *clienteP){
    int i;
    int Acajeros, AcajerosT, AclienteC, AclienteU, AclienteP;
    int conteo=0;

    BorrarPantalla();

    /**Pedir el numero de cajeros***/
    printf("¿Cuántos cajeros quiere que el Banco tenga?\n");
    do{
        scanf("%d", &Acajeros);
        if(Acajeros<1 || Acajeros>10){
            printf("Numero de cajeros no valido (0<N<11).\n");
        }
    }while(Acajeros<1 || Acajeros>10);

    /**Generar cajas abiertas aleatoriamente***/
    for(i=0;i<10;i++){
        cajasB[i]=FALSE;
    }

    srand(time(NULL));
    while(conteo!=Acajeros){
        i=rand()%10;
        if(cajasB[i]==FALSE){
            cajasB[i]=TRUE;
            conteo++;
        }
    }

    /**Pedir tiempo de consulta de los cajeros***/
    printf("¿Cuál es el tiempo de atención de cada cajero?\n");
    do{
        scanf("%d", &AcajerosT);
        if(AcajerosT<10 || AcajerosT%10 != 0){
            printf("Tiempo invalido\n");
        }
    }while(AcajerosT<10 || AcajerosT%10 != 0);

    /**Pedir el tiempo de llegada de los clientes***/
    printf("¿Cuál es el tiempo de llegada de los clientes del banco?\n");
    do{
        scanf("%d", &AclienteC);
        if(AclienteC<10 || AclienteC%10 != 0){
            printf("Tiempo invalido\n");
        }
    }while(AclienteC<10 || AclienteC%10 != 0);

    /**Pedir el tiempo de llegada de los usuarios***/
    printf("¿Cuál es el tiempo de llegada de los usuarios del banco?\n");
    do{
        scanf("%d", &AclienteU);
        if(AclienteU<10 || AclienteU%10 != 0){
            printf("Tiempo invalido\n");
        }
    }while(AclienteU<10 || AclienteU%10 != 0);

    /**Pedir el tiempo de llegada de los preferentes***/
    printf("¿Cuál es el tiempo de llegada de los usuarios preferentes?\n");
    do{
        scanf("%d", &AclienteP);
        if(AclienteP<10 || AclienteP%10 != 0){
            printf("Tiempo invalido\n");
        }
    }while(AclienteP<10 || AclienteP%10 != 0);

    *cajeros = Acajeros;
    *cajerosT = AcajerosT;
    *clienteC = AclienteC;
    *clienteU = AclienteU;
    *clienteP = AclienteP;
}

/*
void Simulacion(int cajeros, int cajerosT, boolean cajasB[], int clienteC, int clienteU, int clienteP)
Recibe:  int Número de cajeros, int Tiempo de atención de los cajeros,
boolean Arreglo de cajas siendo ocupadas, int Tiempo de llegada de un
Cliente,
int Tiempo de llegada de un Usuario, int Tiempo de llegada de un
Preferente
Devuelve: void (No retorna valor explicito)
Observaciones:  Función que simulara el comportamiento de un banco con base en las variables dadas por el
usuario, permitiendo la llegada de clientes, usuarios y preferentes a su
respectiva cola
siendo atendidos en las cajas respetando las normas de un banco siendo
que el banco no
cerrara.
*/
void Simulacion(int cajeros, int cajerosT, boolean cajasB[], int clienteC, int clienteU, int clienteP){
    int tiempo = 0, clientesN[] = {0,0,0}, atendidos = 0;
    int clientesT[] = {clienteC, clienteU, clienteP};
    int controlC=0, controlU=0;

```

```

int i, cajaC, aux;
boolean bucle1, bucle2;
elemento e;

//Inicializar la función Rand
srand(time(NULL));

//Crear filas
cola filas[3];

//Inicializar filas
for(i=0;i<3;i++){
    Initialize(&filas[i]);
}

//Crear las colas
cola cajerosC[cajeros];
int tiempoC[cajeros];
int atendidosC[cajeros];

//Inicializar colas
for(i=0;i<cajeros;i++){
    Initialize(&cajerosC[i]);
    tiempoC[i] = 0;
    atendidosC[i] = 0;
}

//Iniciar Banco
AbrirBanco(cajeros, cajasB);

//Ciclo infinito de la simulación
while(1){
    EsperarMiliSeg(TIEMPO_BASE); //Esperar el tiempo base
    tiempo++; //Incrementar el contador de tiempo

    //Si el tiempo es multiplo del tiempo de atencion de los cajeros
    for(i=0;i<cajeros;i++){
        if ((tiempo+tiempoC[i]) % cajerosT == 0){
            if (!Empty(&cajerosC[i])){
                //Quitar Cliente de la Caja
                e = QuitarClienteCaja(cajasB, &cajerosC[i], i);
                atendidos++;
                atendidosC[i]++;
                MoverCursor(30,4);
                printf("%d", atendidos);
                MoverCursor(62,4);
                printf("C%d - %c%d ", i+1, e.c, e.n);
            }
        }
    }

    //Si el tiempo es multiplo del de llegada de los clientes, usuarios y preferentes
    for(i=0;i<3;i++){
        if(tiempo % clientesT[i] == 0){
            clientesN[i]++; //Incrementar el numero de clientes

            //Agregar Cliente a la Fila
            AgregarClienteFila(&filas[i], clientesN[i], i);
        }
    }

    //Verificar si hay cajas para nuevos clientes, usuarios o preferentes
    bucle1=TRUE;
    while(bucle1){
        //Verificando cajas disponibles
        aux=0;
        for(i=0;i<cajeros;i++){
            if (Empty(&cajerosC[i]))
                aux++;
        }
        if(aux!=0){
            //Verificando filas
            aux=-1;
            if(!Empty(&filas[2])){
                if(aux== -1 && controlC!=2 && controlU!=5){
                    aux=2;
                    controlC++;
                    controlU++;
                }
            }
            if(!Empty(&filas[0])){
                if(aux== -1 && controlU!=5){
                    aux=0;
                    controlU++;
                    controlC = 0;
                }
            }
            if(!Empty(&filas[1])){
                if(aux== -1){
                    aux=1;
                    controlU=0;
                }
            }
        } else {
            controlC = 0;
        }
        if(!Empty(&filas[1])){
            if(aux== -1){
                aux=1;
                controlU=0;
            }
        } else {

```

```

        controlU=0;
    }

    if(aux!=-1){
        bucle2=TRUE;
        while(bucle2){
            cajaC=rand()%cajeros;
            if(Empty(&cajerosC[cajaC])){
                bucle2=FALSE;
            }
            //Agregar Cliente a la Caja
            AgregarClienteCaja(cajasB, &cajerosC[cajaC], &filas[aux], cajaC, aux);
        } else {
            bucle1=FALSE;
        }
    } else {
        bucle1=FALSE;
    }
}

return;
}

/*
void AgregarClienteCaja(boolean cajasB[], cola *cajeroC, cola *filaC, int c, int f)
Recibe: boolean Arreglo de cajas siendo ocupadas, cola * Referencia/Dirección a la cola de un Cajero,
cola * Referencia/Dirección a la cola de una Fila, int Número de Cajero,
int Número de Fila
Devuelve: void (No retorna valor explicito)
Observaciones: Función que Mostrara de manera animada como uno de los tipos de usuario dependiendo del
número
de fila, se pone delante de una caja para ser atendido.
*/
void AgregarClienteCaja(boolean cajasB[], cola *cajeroC, cola *filaC, int c, int f){
    int fila=9, columna=5;
    int i, conteo=0;
    elemento e, aux;

    aux = QuitarClienteFila(filaC, f);

    for(columna=5,i=0;i<10;columna+=7,i++){
        if(cajasB[i]==TRUE){
            if(conteo==c){
                break;
            }
            conteo++;
        }
    }

    MoverCursor(columna,fila);
    if(aux.n<10){
        printf(" %c%d", aux.c, aux.n);
    } else if (aux.n<1000){
        printf(" %c%d", aux.c, aux.n);
    } else {
        printf("%c%d", aux.c, aux.n);
    }

    EsperarMiliSeg(20);
    e.n = aux.n;
    e.c = aux.c;
    Queue(cajeroC, e);
}

/*
elemento QuitarClienteCaja(boolean cajasB[], cola *cajeroC, int c)
Recibe: boolean Arreglo de cajas siendo ocupadas, cola * Referencia/Dirección a la cola de un Cajero, int
Número de Cajero
Devuelve: elemento Elemento
Observaciones: Función que Mostrara de manera animada como con un número de caja dada, el cliente actual se
borrara de está y
se retornara como un elemento.
*/
elemento QuitarClienteCaja(boolean cajasB[], cola *cajeroC, int c){
    int fila=9, columna=5;
    int i, conteo=0;
    elemento e;

    for(columna=5,i=0;i<10;columna+=7,i++){
        if(cajasB[i]==TRUE){
            if(conteo==c){
                break;
            }
            conteo++;
        }
    }

    MoverCursor(columna,fila);
    printf(" ");

    EsperarMiliSeg(20);
    e = Dequeue(cajeroC);
    return e;
}

```

```

/*
void AgregarClienteFila(cola *filaC, int clienteN, int tipo)
Recibe: cola * Referencia/Dirección a la cola de una Fila, int Número de Cliente, int Tipo de cliente
Devuelve: void (No retorna valor explicito)
Observaciones: Función que Mostrara de manera animada como una de las filas se le añade un nuevo cliente,
con base en que tipo de cliente es.
*/
void AgregarClienteFila(cola *filaC, int clienteN, int tipo){
    int fila=12, columna=28+(9*tipo);
    elemento e;
    int tam;

    e.n = clienteN;
    if(tipo==0)
        e.c='C';
    if(tipo==1)
        e.c='U';
    if(tipo==2)
        e.c='P';

    if(Empty(filaC)){
        MoverCursor(columna,fila);
        if(clienteN<10){
            printf(" %c%d", e.c, e.n);
        } else if (clienteN<1000){
            printf(" %c%d", e.c, e.n);
        } else {
            printf("%c%d", e.c, e.n);
        }
    } else {
        tam = Size(filaC);
        if(tam<9){
            MoverCursor(columna,fila+tam);
            if(clienteN<10){
                printf(" %c%d", e.c, e.n);
            } else if (clienteN<1000){
                printf(" %c%d", e.c, e.n);
            } else {
                printf("%c%d", e.c, e.n);
            }
        } else {
            tam-=8;
            MoverCursor(columna,fila+9);
            if(tam<10){
                printf(" +%d", tam);
            } else if (tam<1000){
                printf(" +%d", tam);
            } else {
                printf(" +%d", tam);
            }
        }
    }

    EsperarMiliSeg(20);
    Queue(filaC, e);
}

/*
elemento QuitarClienteFila(cola *filaC, int tipo)
Recibe: cola * Referencia/Dirección a la cola de una Fila, int Tipo de cliente
Devuelve: elemento Elemento
Observaciones: Función que Mostrara de manera animada como con un tipo de cliente, se quitara ese cliente
de esa Fila y se retornara como un elemento.
*/
elemento QuitarClienteFila(cola *filaC, int tipo){
    int fila=12, columna=28+(9*tipo);
    int tam, i;
    elemento e, aux;

    e = Dequeue(filaC);

    MoverCursor(columna,fila);
    if(Empty(filaC)){
        printf(" ");
    } else {
        tam = Size(filaC);

        for(i=0;i<=tam;i++){
            MoverCursor(columna,fila+i);
            printf(" ");
            MoverCursor(columna,fila+i);

            if(i==tam)break;
            if(i==9)break;

            aux = Element(filaC, i+1);
            if(aux.n<10){
                printf(" %c%d", aux.c, aux.n);
            } else if (aux.n<1000){
                printf(" %c%d", aux.c, aux.n);
            } else {
                printf("%c%d", aux.c, aux.n);
            }
        }
    }
}

```

```

        if(tam>9){
            tam-=9;
            if(tam<10){
                printf("  +%d", tam);
            } else if (tam<1000){
                printf(" +%d", tam);
            } else {
                printf("+%d", tam);
            }
        }
    }

    EsperarMiliSeg(10);
    return e;
}

/*
void AbrirBanco(int cajeros, boolean cajasB[])
Recibe:  int Número de Cajeros, boolean Arreglo de cajas siendo ocupadas
Devuelve: void (No retorna valor explicito)
Observaciones:  Función que abre el banco de manera animada en pantalla.
*/
void AbrirBanco(int cajeros, boolean cajasB[]){
    DibujaPresentacion();
    BorrarPantalla();
    DibujaMarco();
    DibujaCajas(cajeros, cajasB);
    DibujaFilas();
    DibujaEstantes();
    DibujaAnuncioAbrir();
}

/*
void DibujaPresentacion()
Recibe:  void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones:  Función que da una presentación a la simulación 03.
*/
void DibujaPresentacion(){
    int columna, fila, i;

    BorrarPantalla();

    //Crear Particulas Aleatorias
    srand(time(NULL));
    EsperarMiliSeg(2200);
    for (i=1;i<=500;i++) {
        columna=(rand()%(ANCHO-1))+1;
        fila=(rand()%(ALTO-1))+1;
        MoverCursor(columna,fila);
        printf("%s");
        if(1%100 == 0){
            EsperarMiliSeg(800);
        }
    }

    //Crear Primer Cuadro
    EsperarMiliSeg(2200);
    for(columna=22;columna<=57;columna++){
        for(fila=4;fila<=13;fila++){
            MoverCursor(columna,fila);
            if((columna>22 && columna<57) && (fila==4 || fila==13)){
                printf(" ");
            } else if ((fila>4 && fila<=13) && (columna==22 || columna==57)){
                printf("|");
            } else {
                printf(" ");
            }
        }
    }

    //Poner datos
    EsperarMiliSeg(1800);
    MoverCursor(28,6);
    printf("Erendil Aguilar Avendaño");
    MoverCursor(28,9);
    printf("Iker Itzae Aguilar Souza");
    MoverCursor(35,12);
    printf("PRESENTAN");

    //Crear Segundo Cuadro
    EsperarMiliSeg(2200);
    for(columna=22;columna<=57;columna++){
        for(fila=17;fila<=20;fila++){
            MoverCursor(columna,fila);
            if((columna>22 && columna<57) && (fila==17 || fila==20)){
                printf(" ");
            } else if ((fila>17 && fila<=20) && (columna==22 || columna==57)){
                printf("|");
            } else {
                printf(" ");
            }
        }
    }
}

```

```

//Poner datos
EsperarMiliSeg(1800);
MoverCursor(34,19);
printf("SIMULACION 03");

EsperarMiliSeg(5000);
BorrarPantalla();
}

/*
void DibujaMarco()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea el todo el marco del Banco así como sus datos iniciales.
*/
void DibujaMarco(){
int columna, fila, i;
int Esperar = 5;

//Crear columnas
for(columna=1, fila=2; fila<ALTO; fila++){
//Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
if(fila<6){
MoverCursor(columna, fila);
printf("|");
MoverCursor(ANCHO-1, fila);
printf("|");
} else {
MoverCursor(columna+1, fila);
printf("||");
MoverCursor(ANCHO-3, fila);
printf("||");
}
EsperarMiliSeg(Esperar);
}

//Crear filas
for(columna=2, fila=1; columna<ANCHO-1; columna++){
//Mover el cursor, dibujar un * y esperar TIEMPO_BASE milisegundos
MoverCursor(columna, fila);
printf(" ");
MoverCursor(columna, fila+4);
printf(" ");
if(columna>3 && columna<ANCHO-3){
MoverCursor(columna, ALTO-1);
printf(" ");
}
EsperarMiliSeg(Esperar);
}

//Crear texto
MoverCursor(27,3);
printf("\nBanco Nacional de México\n");
EsperarMiliSeg(200);
MoverCursor(5,4);
printf("Num. Clientes Atendidos: 0");
EsperarMiliSeg(200);
MoverCursor(42,4);
printf("Ultimo movimiento:");
EsperarMiliSeg(200);
}

/*
void DibujaCajas(int cajeros, boolean cajasB[])
Recibe: int Número de Cajeros, boolean Arreglo de cajas siendo ocupadas
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea las cajas que usara el Banco con base a un número de cajas
dado y con un arreglo que cajas usar.
*/
void DibujaCajas(int cajeros, boolean cajasB[]){
int columna=4, fila=6, i, j;
int Esperar = 5;
int conteo = 1;

//Crear Cajas
for(columna=4, i=1, fila=6; i<=10; columna+=7, i++){
//***Crear Cajas***//
//Lineas
for(j=1; j<=6; j++){
MoverCursor(columna+j, fila+2);
printf("_");
if(i==10){
printf(" ");
}
EsperarMiliSeg(Esperar);
}
for(j=0; j<5; j++){
if(j==3 || j==4){
MoverCursor(columna, fila+j);
if(i==1){
printf(":");
} else {
printf("-");
}
}
if(i==10){

```

```

        MoverCursor(columna+8, fila+j);
        printf(":");
    } else {
        MoverCursor(columna+7, fila+j);
        printf("-");
    }
    continue;
}
MoverCursor(columna, fila+j);
printf("|");
MoverCursor(columna+(i==10?8:7), fila+j);
printf("|");
EsperarMiliSeg(Esperar);
}
if(cajasB[i-1]){
    MoverCursor(columna+3, fila+1);
    printf("C%d", conteo);
    conteo++;
} else {
    for(j=0; j<2; j++){
        MoverCursor(columna+1, fila+j);
        printf("////");
        if(i==10){
            printf("/");
        }
    }
}
}

}

/*
void DibujaFilas()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que crea las Filas que usaran los 3 tipos de clientes.
*/
void DibujaFilas(){
    int columna=25, fila=11, i, j;
    int Esperar = 5;

    //Crear Cajas
    for(columna=25, i=1, fila=11; i<=4; columna+=9, i++){
        /***Crear Lineas***/
        for(j=0; j<11; j++){
            if(j==0){
                MoverCursor(columna, fila+j);
                printf(" _ ");
                EsperarMiliSeg(Esperar);
                continue;
            }
            if(j==10){
                MoverCursor(columna, fila+j);
                printf(" |_| ");
                EsperarMiliSeg(Esperar);
                continue;
            }
            MoverCursor(columna, fila+j);
            printf("|||");
            EsperarMiliSeg(Esperar);
        }
    }
}

/*
void DibujaEstantes()
Recibe: void (No recibe valor explicito)
Devuelve: void (No retorna valor explicito)
Observaciones: Función que dibuja Estantes de decoración para la simulación.
*/
void DibujaEstantes(){
    int columna=6, fila=12, i, j;
    int Esperar = 5;
    int conteo = 1;

    //Crear Cajas
    for(columna=6, i=1, fila=12; i<=5; fila+=2, i++){
        /***Crear Estantes***/
        if(i%2==0){
            MoverCursor(columna, fila);
            printf("[$$$$$$$$$$]");
            MoverCursor(ANCHO-20, fila);
            printf("[$$$$$$$$$$]");
        } else {
            MoverCursor(columna, fila);
            printf("[#####]");
            MoverCursor(ANCHO-20, fila);
            printf("[#####]");
        }
        EsperarMiliSeg(Esperar);
    }
}

/*
void DibujaAnuncioAbrir()
Recibe: void (No recibe valor explicito)

```



```

    Devuelve: void (No retorna valor explicito)
    Observaciones: Función que da un anuncio en pantalla de que el banco va a abrir.
*/
void DibujaAnuncioAbrir(){
    int columna=39, fila=16, i;
    int Esperar = 800;

    //Escribir Conteo
    for(i=3;i>0;i--){
        MoverCursor(columna,fila);
        EsperarMiliSeg(Esperar);
        printf("%d", i);
    }
    EsperarMiliSeg(Esperar);

    //Poner GO
    MoverCursor(columna,fila);
    printf("GO");
    EsperarMiliSeg(Esperar);

    //Quitar GO
    MoverCursor(columna,fila);
    printf(" ");
    EsperarMiliSeg(Esperar);
}

```

Bibliografía

[1] E. A. Franco Martínez "2.3. Cola", Notas de clase de ALG. Y EST. DE DATOS 24-2, Ingeniería en Sistemas Computacionales, Escuela Superior de Computo, primavera 2024.

[2] E. A. Franco Martínez "Práctica 04: Simulaciones con el TADCola", Notas de clase de ALG. Y EST. DE DATOS 24-2, Ingeniería en Sistemas Computacionales, Escuela Superior de Computo, primavera 2024.